

People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research
University M'Hamed BOUGARA – Boumerdes



Institute of Electrical and Electronic Engineering
Department of Electronics

Final Year Project Report Presented in Partial Fulfilment of
the Requirements for the Degree of

MASTER

In Electrical and Electronics Engineering
Option: Computer Engineering

Title:

**Diabetic Retinopathy Grading using Deep
Neural Networks**

Presented by:

Amira TEMMAM

Chaima AHMED GAID

Supervisor:

Prof. Abdelhamid DAAMOUCHE

Co-Supervisor:

Dr. Abbas CHEDDAD

Registration Number:...../2023

Abstract

Diabetic Retinopathy (DR) is a chronic disease and the leading cause of blindness and visual impairment among diabetic patients making early detection and classification of DR crucial for effective treatment. This project utilizes state-of-the-art convolutional neural networks and transfer learning techniques to analyze retinal images and classify the severity of the disease on a scale of 0 to 4 (ranging from healthy to proliferative). We employ architectures such as EfficientNetB1, InceptionV3, Xception, and MobileNetV2 to achieve accurate classification. Through a series of experiments and evaluations, our system achieves an overall accuracy of 80.0% in classifying DR images. The results showcase the significant potential of deep learning in assisting healthcare professionals with early diagnosis and treatment planning, thereby improving patient well-being.

Keywords: Diabetic retinopathy grading, diabetic retinopathy detection, deep learning, convolutional neural networks, transfer learning, retinal fundus images.

Dedication

All praise to **Allah**, the Lord of the worlds, for His blessings and guidance throughout this journey

First I would like to thank my **Teapot** for never failing to keep me awake during the long nights of my academic journey.

I would like to dedicate this project to my incredible **Parents**, who have always been the guidance stars of my life, and to my remarkable **Mother**, who is also graduating alongside me. Your love, support, and tireless dedication to my education have been my driving force.

To my awesome siblings and my “best-teas”, **Maha, Selma, Manel**, and my big brother **Moussa** who have stood by me through thick and thin. Thank you for being my partners in crime and the extra teaspoon of fun in my life.

To my **extended family**, you hold a special place in my heart. Your presence, wisdom, and unconditional love have shaped my character and provided a strong foundation for me to pursue my dreams.

To my dearest friends **Amani, Roumaissa, Nada, Chichi, Bouchra, Nabila, Manel, Lidia, Cylia**, and to my partner and friend **Chaima**, your friendship has enriched my life in countless ways, and I am grateful for the bond we have forged.

To my **college friends**, you have been more than just classmates From late-night study sessions to hilarious memes, we have shared laughter and unforgettable memories.

In loving memory of my late **Khali Abderrahmen**, this dissertation is dedicated to your enduring memory

Amira

This work is dedicated to:

The sake of **Allah**, my Creator and my Master, for helping and guiding me through all the way,

My great teacher and messenger, **Mohammed** peace be upon him, who taught me the purpose of life,

My beloved **parents**, who have been my constant source of support, inspiration, and unwavering courage, kept me moving forward at all times and set me on this journey early in life,

My **grandmother** for her valuable prayers and encouragements during my studies,

My dearest sister **Zohra** who has been guiding me through my research experience, and my brother **Imed** who believed in my potential and motivated me to give my utmost effort,

My sisters **Maha**, **Soumia**, and **Hanane** for their valuable support and positive influence,

My dearest uncle **Mabrouk Hammadi** who always believed in me,

My lifelong friends **Meissa** and **Rayane**,

All my cherished friends with whom I shared the invaluable journey of university, **Manar**, **Cheima**, **Cylia**, **Nabila**, **Zehor**, **Aya**, **Amani**, **Roumaissa**, **Amel**, **Nour** and of course my partner **Amira**.

And everyone who gave me their advice and encouragement to finish this work.

Chaima

Acknowledgements

First and foremost, we praise **Allah** the Almighty and most merciful for His blessings, knowledge, and for granting us the wisdom, persistence, and capability to successfully complete this project. We hold great reverence for the patronage and moral support bestowed upon us by our families, especially our parents, whose financial contributions and unwavering encouragement have made this endeavor a reality.

We extend our sincere appreciation to our supervisor, **Prof. Abdelhamid DAAMOUCHE**, for his support and guidance throughout this project. His feedback, constructive criticism, and constant encouragement have been invaluable in shaping the direction and enhancing the quality of our work.

We would also like to express our gratitude to **Dr. Abbas CHEDDAD** for inspiring us to embark on this project, which holds immense significance for us. Additionally, we extend our heartfelt appreciation to our internship supervisors, **Mr. Belkacem ELHACHEMI** and **Mrs. Fatiha ALIM** at CDTA, for their support and mentorship during our internship period.

Last but certainly not least, we would like to thank **Dr. Yasmine MAKHLOUF** for her support and encouragement, as well as all the members of **IEEE Student Branch** University of Boumerdes for accompanying us on this challenging and memorable journey. We would like to express our special thanks to **Moussa GUENANI** for his assistance and inspiration, and we wish him the best of luck in his future endeavors.

Table of Contents

Abstract	i
Dedication	ii
Acknowledgements	iv
List of Tables	ix
List of Figures	x
List of Abbreviations	xiii
List of Abreviations	xiii
General Introduction	1
1 Diabetic Retinopathy Overview	4
1.1 Introduction	4
1.2 The Human Eye Anatomy	4
1.3 Retina	5
1.4 Diabetic Retinopathy	6
1.5 Stages of Diabetic Retinopathy	7
1.6 Fundus Photography	7
1.7 Conclusion	8

2	Deep Learning Background and Related Work	9
2.1	Introduction	9
2.2	Machine Learning	9
2.2.1	Machine Learning General Model	10
2.2.2	Types of Machine Learning	11
2.2.3	Deep Learning	11
2.3	Neural Network Training	13
2.3.1	Loss function	14
2.3.2	Gradient descent	14
2.3.3	Backward propagation	14
2.3.4	Dropout	15
2.3.5	Batch normalization	15
2.3.6	Hyperparameters	16
2.3.7	Activation Functions	17
2.4	Convolutional Neural Networks	20
2.4.1	Convolutional Layer	20
2.4.2	Pooling Layer	21
2.4.3	Fully Connected Layer	22
2.5	Related Work	22
2.6	Conclusion	27
3	Approach and Implementation	28
3.1	Introduction	28
3.2	Workflow	28
3.3	Tools	29
3.3.1	Python	29
3.3.2	TensorFlow	29
3.3.3	Keras	29

3.3.4	Scikit-learn	30
3.3.5	OpenCV	30
3.3.6	Kaggle	30
3.4	Datasets	30
3.4.1	EyePACS	30
3.4.2	APTOS	31
3.4.3	DDR	31
3.5	Pre-processing	32
3.5.1	Data Refinement	32
3.5.2	Image Enhancement	34
3.5.3	Circular Cropping	35
3.6	Data augmentation	36
3.6.1	Traditional Augmentation Techniques	37
3.6.2	Augmentation using Morphological Filters	38
3.7	Evaluation metrics	39
3.7.1	Confusion matrix	39
3.7.2	Accuracy	40
3.7.3	Precision	40
3.7.4	Sensitivity (Recall)	40
3.7.5	F1-score	41
3.7.6	Weighted average	41
3.8	Transfer learning	41
3.8.1	EfficientNetB1	41
3.8.2	InceptionV3	42
3.8.3	Xception	43
3.8.4	MobileNetV2	44
3.9	Conclusion	45

4	Results and Discussion	46
4.1	Introduction	46
4.2	Phase 1: Initial Training	46
4.3	Phase 2: Experiments on EfficientNetB1	49
4.3.1	EfficientNetB1 Experiment I	50
4.3.2	EfficientNetB1 Experiment II	52
4.3.3	EfficientNetB1 Experiment III	54
4.4	Phase 3	56
4.4.1	Experiment on InceptionV3	56
4.4.2	Experiment on Xception	59
4.4.3	Experiment on MobileNetV2	61
4.5	Phase 4: Applying Morphological Operations	64
4.6	Phase 5: Binary Classification	66
4.7	Summary of Results	70
4.8	Discussion	70
4.9	Conclusion	72
	General Conclusion	73
	Bibliography	75
	APPENDICES	75

List of Tables

2.1	Comparison between simple and multilayer perceptron	13
2.2	Diabetic Retinopathy detection and grading studies	26
3.1	Distribution of EyePACS, APTOS, and DDR datasets	32
3.2	Parameter settings for augmentation	37
4.1	Selected hyperparameter values	48
4.2	Classification report on EfficientNetB1's first experiment	51
4.3	Classification report of EfficientNetB1 experiment II	53
4.4	Classification report of EfficientNetB1 experiment III	55
4.5	Classification report of InceptionV3 model	58
4.6	Classification report on Xception experiment	60
4.7	Classification report on MobileNetV2 experiment	63
4.8	Classification report on InceptionV3 using morphological operations	65
4.9	Classification report of the four models for binary classification	69
4.10	Summary of results and overall comparison among experiments	70

List of Figures

1	Inconsistency of ophthalmologists' estimation	2
1.1	Human eye anatomy	5
1.2	Fundus of the eye showing the macula, fovea, and optic disc	5
1.3	Normal eye vs Diabetic eye	6
1.4	Stages of Diabetic Retinopathy	7
1.5	Fundus photography color imaging	8
2.1	Components of a machine learning model	10
2.2	Biological neuron schematics	12
2.3	Single perception	12
2.4	Multilayer perception	13
2.5	Gradient descent and weights update in Backpropagation algorithm	15
2.6	Dropout Regularization	15
2.7	Sigmoid activation function	18
2.8	ReLU activation function	19
2.9	A representative structure of CNN	20
2.10	Convolution operation on a 6x6x3 image matrix with a 4x4x3 KernelR . .	21
2.11	Max-pooling layer operation	22
2.12	Types of diabetic retinopathy studies	23
2.13	Backbones used for diabetic retinopathy detection studies	25
3.1	General block diagram of the proposed work	29

3.2	Examples of overexposed and underexposed images from the dataset	33
3.3	Unfit images in the dataset	33
3.4	Retinal image preprocessing	35
3.5	Circle cropped retinal images	36
3.6	Bar graph of the final dataset distribution	37
3.7	Morphological operation on a retinal image	38
3.8	An illustrative example of the confusion matrix for a multi-class classification test	39
3.9	Architecture of EfficientNetB1 with MBConv as basic building blocks . . .	42
3.10	Structure of an inverted residual block (MBconv)	42
3.11	InceptionV3 architecture	43
3.12	Modified depth-wise separable convolution for Xception architecture	44
3.13	MobileNet architecture	44
4.1	Confusion matrix of the first training on AlexNet and VGG16 models . . .	47
4.2	Loss and accuracy graphs of AlexNet model.	47
4.3	Loss and validation graphs of the VGG16 model.	48
4.4	EfficientNetB1 model summary	50
4.5	Loss and validation graphs of the EfficientNetB1 model during experiment I	50
4.6	Confusion matrix of EfficientNetB1 during experiment I	51
4.7	Loss and validation graphs of EfficientNetB1 model during experiment II .	52
4.8	Confusion matrix of EfficientNetB1 during experiment II	53
4.9	Loss and validation graphs of EfficientNetB1 model during experiment III	55
4.10	Confusion matrix of EfficientNetB1 during experiment III	56
4.11	InceptionV3 model summary	57
4.12	Loss and validation graphs of InceptionV3 model	57
4.13	Confusion matrix of InceptionV3 model	58
4.14	Xception model summary	59

4.15	Loss and validation graphs of Xception model	60
4.16	Confusion matrix of Xception model	61
4.17	MobileNetV2 model summary	62
4.18	Loss and validation graphs of MobileNetV2 model	62
4.19	Confusion matrix of MobileNetV2 model	63
4.20	Loss and validation graphs of InceptionV3 model with morphological filters augmentation	64
4.21	Confusion matrices before and after applying morphological operations . .	65
4.22	Binary class distribution	66
4.23	Confusion matrices of the four models for binary classification	67
4.24	Loss and validation graphs of MobileNetV2 model for binary classification	68

List of Abbreviations

DR	Diabetic Retinopathy.
AI	Artificial Intelligence.
ML	Machine Learning.
DL	Deep Learning.
ANN	Artificial Neural Network.
CNN	Convolutional Neural Networks.
ReLU	Rectified Linear Unit.
GPU	Graphics Processing Unit.

General Introduction

The global prevalence of Diabetes Mellitus is a major health concern, affecting millions of individuals worldwide, with projections estimating a rise to 700 million cases by 2045 [1]. Diabetes occurs when the body fails to produce enough insulin or properly utilize the insulin it generates. Poor management of blood sugar levels and the duration of the disease increase the risk of complications, including kidney damage, nerve damage, eye damage, cardiovascular disease, hearing loss, Alzheimer's, skin issues, infections, foot damage, and other related ailments.

One of the common complications associated with diabetes is Diabetic retinopathy (DR), an ocular condition that can lead to visual impairment and eventual blindness. The prevalence of DR has reached alarming levels globally, yet there is currently no effective treatment available. However, studies have shown that early detection and timely treatment of diabetic retinopathy can significantly reduce the risk of blindness. Regular screening programs, such as annual eye examinations, are recommended by national health agencies to mitigate DR-induced blindness. Digital fundus images have become the preferred imaging modality for ophthalmologists in screening and assessing the severity of DR, providing valuable insights into the disease's progression. However, the recommendation of annual eye examinations faces practical challenges in many countries. Firstly, a significant number of patients, particularly in developing countries, cannot afford routine eye tests. Secondly, the distinct characteristics of each stage of diabetic retinopathy present challenges for physicians in making accurate diagnoses. Even highly skilled clinicians may struggle to assess the stages of diabetic retinopathy accurately through manual evaluation. This limitation was highlighted by a study conducted by Google [2] as shown in Figure 1. Consequently, there is growing interest in the concept of automatic detection as a means to improve the accuracy and efficiency of diagnosing diabetic retinopathy.

Recent advancements in Artificial Intelligence (AI) and the availability of increased com-

putational resources have paved the way for the development of Deep Learning (DL) applications in accurate detection and classification of Diabetic Retinopathy (DR). The need for a comprehensive and automated DR screening method has long been acknowledged, and previous efforts utilizing image classification, pattern recognition, and machine learning have shown progress. However, current research on AI-based DR diagnosis still requires improvement in terms of diagnostic accuracy for each stage, as many deep learning algorithms have been trained on small datasets, limiting their ability to accurately predict different stages of diabetic retinopathy. Deep learning approaches, which have excelled in various computer vision and image analysis tasks, have proven highly competitive and offer promising potential in automating the analysis of retinal fundus images for detecting diabetic retinopathy. Specifically, Convolutional Neural Networks (CNNs) have demonstrated their superiority in this field, showcasing their proficiency in the classification and detection of problems related to diabetic retinopathy.

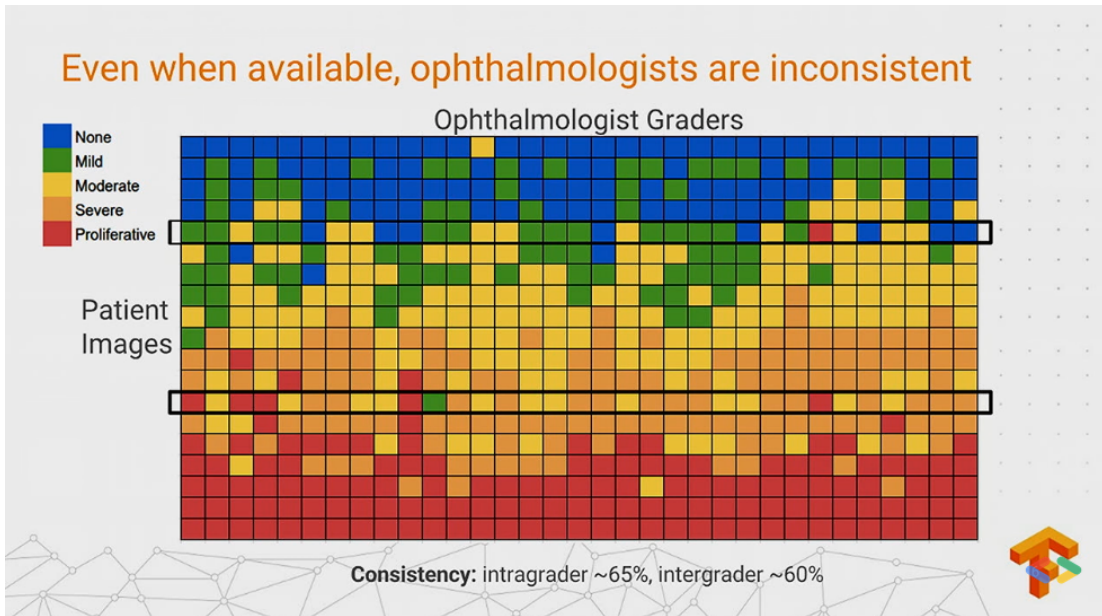


Figure 1: Inconsistency of ophthalmologists' estimation [3]

The motivation of this work is to design a system that can automatically detect and diagnose Diabetic Retinopathy (DR) from eye fundus images using Deep Learning image classification architectures with transfer learning. We use three publicly accessible datasets, namely EyePACS, APTOS, and DDR, for training and testing four pre-trained models based on CNN: EfficientNetB1, InceptionV3, Xception, and MobileNetV2. We evaluate the transfer learning models using several evaluation metrics, including accuracy, weighted average, recall, precision, and F1 score.

The main objectives of this work are as follows:

- Precisely grade Diabetic Retinopathy with a focus on enhancing detection in the Mild and Severe classes.
- Test the model on a single dataset for consistency and comparability.
- Enhance the robustness of the model by merging different datasets to challenge its performance.
- Train and evaluate state-of-the-art transfer learning models on the combined dataset for Diabetic Retinopathy classification.
- Conduct an extensive evaluation of four factors relevant to deep convolutional neural networks for DR classification: CNN architectures, image preprocessing techniques, augmentation methods for addressing class imbalance, and hyperparameter tuning.

The details of this thesis are well explained in the upcoming chapters, which are organized as follows:

- Chapter 1: provides an overview of human eye anatomy, explores the background of Diabetic Retinopathy (DR), and examines the role of Fundus photography in diagnosing and monitoring DR stages.
- Chapter 2: covers the fundamentals of machine learning and deep learning, including artificial neural network architecture, training techniques, and convolutional neural networks. It also reviews the literature and research done before.
- Chapter 3: presents the project's approach and implementation details, discussing the workflow, tools, datasets, preprocessing techniques, and transfer learning using specific architectures.
- Chapter 4: focuses on the outcomes and discussion of the study, evaluating the performance of transfer learning models in detecting DR stages.

Chapter 1

Diabetic Retinopathy Overview

1.1 Introduction

Understanding the anatomy and physiology of the human eye is important for developing effective diagnostic and medical approaches for ocular diseases. Among these diseases, Diabetic Retinopathy (DR) stands out as a significant threat to vision, especially for individuals living with diabetes. This chapter aims to provide an overview of human eye anatomy, delve into the background of DR and explore the role of Fundus photography in diagnosing and monitoring the stages of DR.

1.2 The Human Eye Anatomy

The human eye is a complex organ responsible for our sense of vision, it comprises various components that work together seamlessly to capture, focus, and transmit visual information to the brain. These layers of tissue and structures include the sclera, fovea, iris, macula, lens, optic nerve, pupil, choroid, retina, and vitreous humor . Each of the structures plays a crucial role in the process of vision, and abnormalities in any of these structures can lead to visual impairment [4].

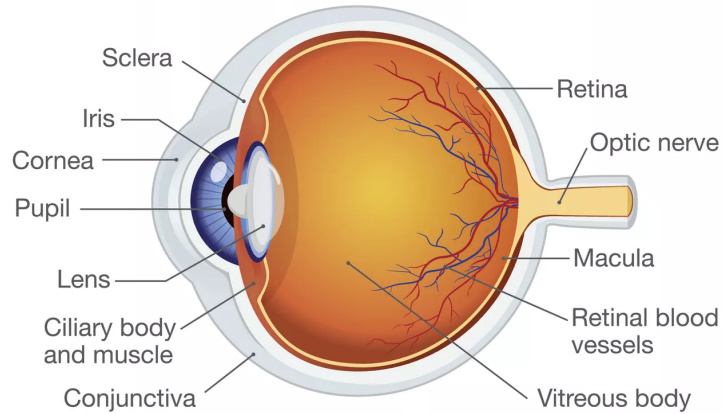


Figure 1.1: Human eye anatomy [5]

1.3 Retina

The retina is a vital component of the eye responsible for capturing and transmitting visual information to the brain. It is a thin, transparent layer of tissue located at the back of the eye responsible for converting light into neural signals that can be interpreted by the brain. It is composed of several layers of specialized cells, including photoreceptor cells (rods and cones) that capture light. The photoreceptors convert light to electrical signals which are further processed by other retinal cells before being sent to the brain through the optic nerve [6]. Figure 1.2 shows the most relevant features in the retina. These are blood vessels, macula, fovea, and optic disc.

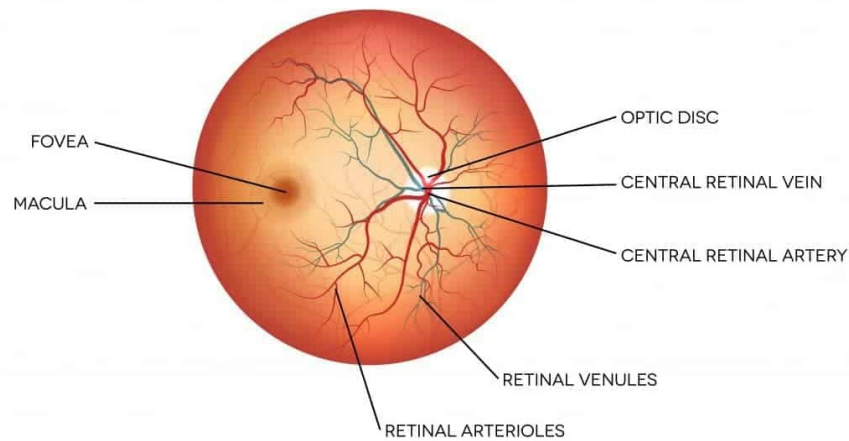


Figure 1.2: Fundus of the eye showing the macula, fovea, and optic disc [7]

- **Blood vessels:** in the retina play a crucial role in maintaining the health and function of retinal cells. The retinal vasculature is composed of several layers of small blood vessels that supply oxygen and nutrients to the retinal tissue.
- **Macula:** is a small, specialized area in the center of the retina that is responsible for high-resolution vision, such as reading and recognizing faces.
- **Fovea:** is a small, central area of the retina that is responsible for high visual acuity and color vision. It contains a high concentration of cone cells, which are responsible for detecting fine details and color.
- **Optic disc:** is an important feature of the retina, it is the point at which the optic nerve leaves the eye, and it appears as a circular area that is slightly lighter in color than the surrounding retina [5, 8].

1.4 Diabetic Retinopathy

Diabetic Retinopathy (DR) is a major cause of blindness worldwide, affecting approximately one-third of diabetic patients [9]. It occurs due to prolonged exposure to high blood sugar levels, which can damage the blood vessels that nourish the retina leading to vision loss or blindness if left untreated. Early detection and timely treatment are crucial to prevent or delay the progression of DR [10].

The detection of DR and its current stage can be determined by investigating the presence or absence of several lesions. Some of the lesions are microaneurysms, hemorrhages, hard exudates, cotton wool spots, and abnormal growth of blood vessels. Figure 1.3 shows a comparison between a healthy and an unhealthy retina.

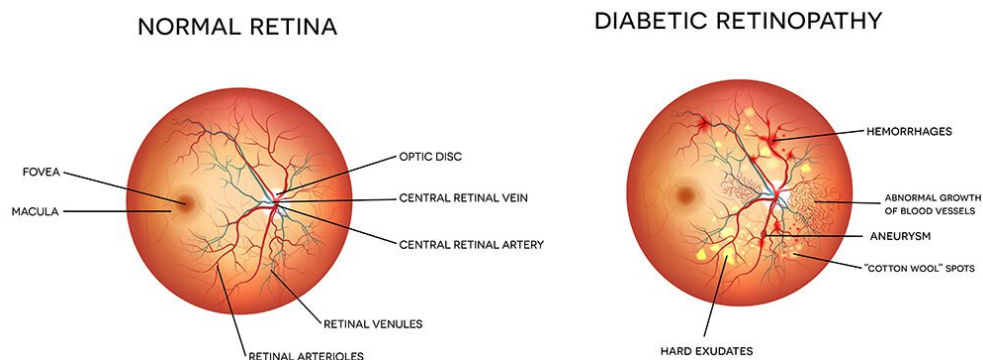


Figure 1.3: Normal eye vs Diabetic eye [11]

1.5 Stages of Diabetic Retinopathy

There are four stages and two forms of diabetic retinopathy. Nonproliferative and proliferative are the two categories. Early disease phases are referred to as nonproliferative (mild, moderate, and severe stages), the blood vessels in the retina may leak fluids and blood, leading to the formation of deposits and swelling while advanced disease stages are referred to as proliferative (proliferate DR stage) where the retina responds to the damage by growing new abnormal blood vessels. These vessels are fragile and prone to leakage, causing further damage and potential vision loss [12, 11]. The four phases are illustrated in Figure 1.4.

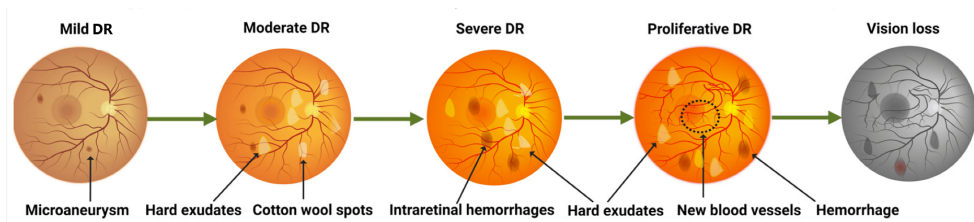


Figure 1.4: Stages of Diabetic Retinopathy [12]

Stage 1: Mild nonproliferative DR, indicated by minute swellings in the retinal blood vessels.

Stage 2: Moderate nonproliferative DR, characterized by retinal blood vessel blockages.

Stage 3: Severe nonproliferative DR, distinguished by a sharp decline in retinal blood flow and the development of new blood vessels.

Stage 4: Proliferative DR, characterized by the growth of new, flimsy blood vessels that are prone to fluid leakage in the retina.

1.6 Fundus Photography

Regular screening for diabetic retinopathy is essential for early detection and timely intervention. Various screening methods are employed to assess the presence and severity of diabetic retinopathy such as Optical Coherence Tomography (OCT), Fundus Photography, and Fluorescein Angiography (FA). Among these, fundus photography is a widely used technique that involves capturing high-resolution images of the retina. These images can reveal abnormalities such as microaneurysms, hemorrhages, and exudates, which are indicative of diabetic retinopathy. Changes or abnormalities in the retina can be visualized

using fundus photography with a field of view of 40 degrees or more [13].

Color Fundus Retinal Photography is a technique that utilizes a specialized camera known as a fundus camera or retinal camera to capture color images of the interior surface of the eye (Figure 1.5). This procedure is aimed at documenting the presence of eye disorders and monitoring their progression over time. By imaging the retina, this technique helps to document conditions such as diabetic retinopathy, age-related macular degeneration, macular edema, and retinal detachment.

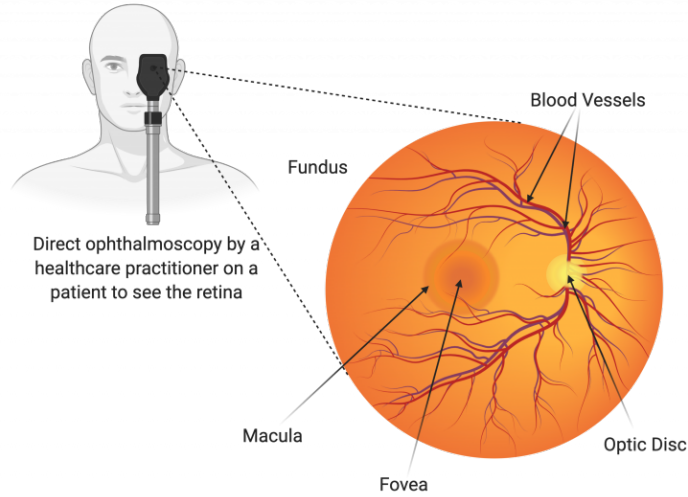


Figure 1.5: Fundus photography color imaging [6]

1.7 Conclusion

In this chapter, we have explored the human eye anatomy, emphasizing its crucial role in visual perception. We have delved into the complexities of the retina and its significance in diabetic retinopathy. Understanding the stages of this progressive disease has underscored the importance of early detection and intervention. Fundus photography and other screening methods have been discussed as essential tools for accurate diagnosis. In the upcoming chapter, we will delve into a brief introduction to Deep Learning technologies and a theoretical overview of Convolutional Neural Networks architecture that enable effective classification of diabetic retinopathy, paving the way for improved diagnostic and management strategies.

Chapter 2

Deep Learning Background and Related Work

2.1 Introduction

The field of machine learning has witnessed significant advancements, revolutionizing various industries including healthcare. Deep learning, a subset of machine learning, has shown remarkable capabilities in analyzing complex medical data and extracting valuable insights. In this chapter, we will go through the fundamentals of machine learning and deep learning, discuss the architecture of artificial neural networks, explore training techniques, activation functions, convolutional neural networks layers, and review related work in the field.

2.2 Machine Learning

Machine learning is a subfield of Artificial Intelligence (AI) that focuses on the development of algorithms and models that enable computers to learn from data and make predictions or take actions without being explicitly programmed for each specific task. And According to Andrew Ng, a renowned computer scientist and one of the pioneers of modern machine learning: "Machine learning is the science of getting computers to learn and act like humans do, and improve their learning over time in an autonomous fashion, by feeding them data and information in the form of observations and real-world interactions."

2.2.1 Machine Learning General Model

The general model for machine learning typically involves the following steps:

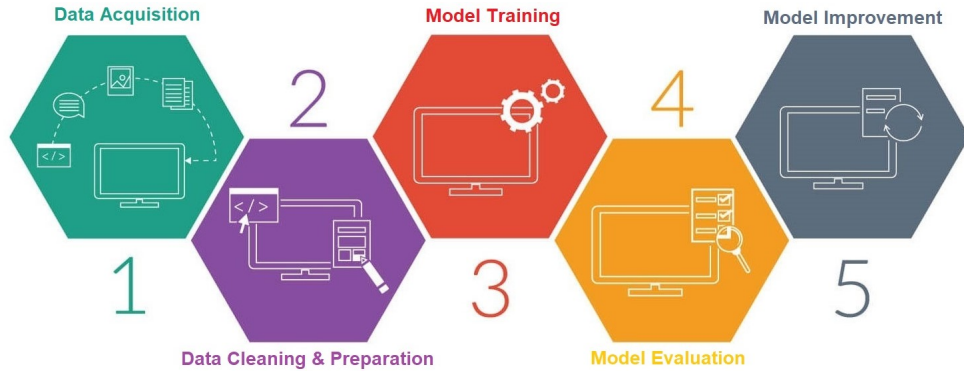


Figure 2.1: Components of a machine learning model

- **Data Acquisition:** The first step is to gather relevant data that will be used to train and evaluate the machine learning model. This data can come from various sources, such as databases, APIs, or sensors.
- **Data Cleaning and Preparation:** Once the data is acquired, it often needs to be cleaned and preprocessed. This involves handling missing values, dealing with outliers, normalizing or scaling features, and transforming the data into a suitable format for further analysis.
- **Model Training:** After the data is prepared, a machine learning model is selected and trained using the cleaned and prepared data. The model learns from the input data and adjusts its internal parameters or structure to capture patterns and relationships.
- **Model Evaluation:** Once the model is trained, it needs to be evaluated to assess its performance and generalization ability. Evaluation is typically done using a separate set of data called the test set, which was not used during training. Common evaluation metrics include accuracy, precision, recall, F1 score, or mean squared error, depending on the specific learning task.
- **Model Improvement:** Based on the evaluation results, the model can be fine-tuned or optimized to improve its performance. This may involve adjusting hyperparameters (e.g., learning rate, regularization strength), trying different algorithms or architectures, or collecting additional data to address any identified limitations or shortcomings [14].

2.2.2 Types of Machine Learning

Machine learning algorithms can be broadly categorized into three types: Supervised Learning: In supervised learning, the algorithm is trained on labeled data, where each example is associated with a known target or outcome. The algorithm learns to map input features to the desired output based on the provided examples. It can then make predictions on new, unseen data by applying the learned mapping. Unsupervised Learning: Unsupervised learning deals with unlabeled data, where the algorithm aims to find inherent patterns or structures in the data without any specific target or outcome to predict. Clustering, dimensionality reduction, and anomaly detection are common tasks in unsupervised learning. Reinforcement Learning: Reinforcement learning involves an agent interacting with an environment and learning to make decisions or take actions to maximize a reward signal. The agent learns through trial and error, receiving feedback from the environment in the form of rewards or penalties for its actions. The goal is to find an optimal policy that maximizes the cumulative reward over time.

2.2.3 Deep Learning

Deep Learning in Healthcare

Deep learning has revolutionized healthcare by enabling advanced analysis of medical data and improving disease diagnosis and treatment. With its ability to automatically learn complex patterns from large datasets, deep learning algorithms have been applied to various healthcare domains, including medical imaging, genomics, electronic health records, and drug discovery. For instance, deep learning models have demonstrated exceptional performance in tasks such as image classification, segmentation, and detection, leading to more accurate diagnoses and personalized treatment plans. One notable example is the use of deep learning in diagnosing diabetic retinopathy, where convolutional neural networks have shown remarkable success in grading disease severity from retinal images [15]. By leveraging the power of deep learning, healthcare professionals can benefit from enhanced decision support systems, improved patient outcomes, and more efficient healthcare delivery.

Artificial Neural Networks

Artificial Neural Networks (ANN) s are computational models that consist of interconnected nodes or artificial neurons organized in layers. There are two types of neurons

involved in ANNs: the biological neurons found in the human brain, which serve as the inspiration, and the artificial neurons, also known as perceptrons, that mimic their behavior in the ANN architecture.

- **The biological neuron**

The biological neuron is the fundamental unit of the human brain's neural network, comprising a cell body, dendrites, an axon, and synapses. Neurons interconnect to transmit chemical and electrical signals, with dendrites receiving information and axons transmitting it. Synapses play a vital role in determining the strength of connections between neurons [16]. Figure 2.2 illustrates the schematic of a biological neuron.

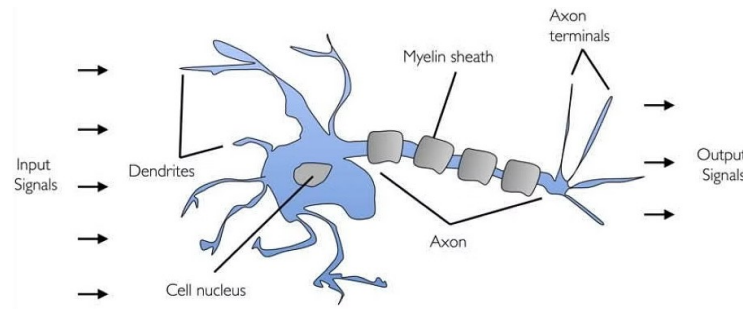


Figure 2.2: Biological neuron schematics[16]

- **Artificial neuron**

Also called perceptron, emulate the functionality of a biological neuron in ANNs. It receives multiple inputs, each multiplied by a synaptic weight, and calculates the weighted sum. Adding a bias term and applying an activation function introduces nonlinearity, enabling ANNs to capture complex relationships. The output of an artificial neuron serves as input for subsequent neurons in the network, facilitating information flow and learning [17, 18]

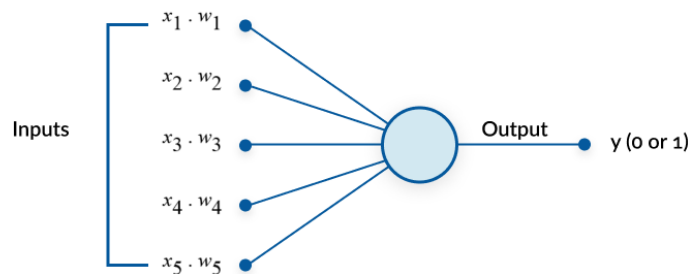


Figure 2.3: Single perception [18]

- **Multilayer perceptron**

Multilayer perceptron (MLP) is a more powerful and flexible ANN architecture that consists of multiple hidden layers between the input and output layers as the diagram in Figure 2.4 shows. Each neuron in the hidden layers applies an activation function to the weighted sum of its inputs. MLPs can learn complex nonlinear relationships between inputs and outputs, making them suitable for a wide range of tasks.

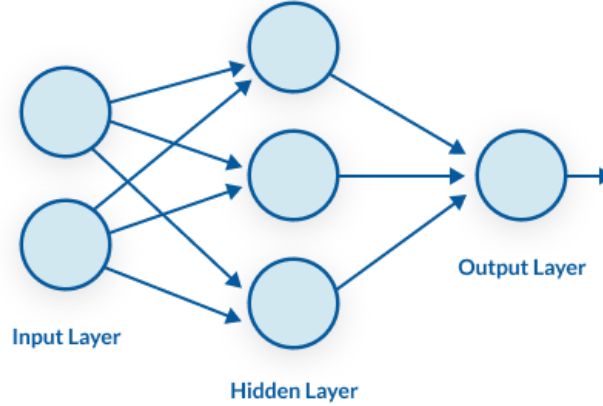


Figure 2.4: Multilayer perception [18]

The comparison between simple and multilayer perceptron can be demonstrated in Table 2.1

Biological Neuron	Artificial Neuron
Nucleus sama	Cell
Dendrites	Input
Synapse	Weights
Axon	Output

Table 2.1: Comparison between simple and multilayer perceptron

2.3 Neural Network Training

Training a neural network involves the iterative process of adjusting its parameters to minimize a chosen loss function. This process is crucial for enabling the network to generalize

well to unseen data and perform effectively on various tasks.

2.3.1 Loss function

A loss function, also known as a cost function, takes into account the probabilities or uncertainty of a prediction based on how much the prediction varies from the true value. This gives us a more nuanced view into how well the model is performing.

2.3.2 Gradient descent

Gradient descent is an optimization algorithm that updates the network's parameters (weights and biases) iteratively. It uses the gradient of the loss function (L) with respect to the parameters to determine the direction and magnitude of updates. The parameters are adjusted as:

$$\theta = \theta - learningrate * \Delta L \quad (2.1)$$

where θ represents the network's parameters, and the learning rate controls the step size in each iteration.

2.3.3 Backward propagation

Backpropagation is an algorithm used to calculate the gradients of a neural network by propagating errors backward through the network. It involves computing the partial derivatives of the loss function with respect to the network's parameters using the chain rule of calculus. The gradients are then used to update the parameters during training via gradient descent. Mathematically, during backpropagation, the gradients (Δ) of the loss function (L) with respect to the parameters (θ) are calculated as:

$$\Delta = \frac{dL}{d\theta} \quad (2.2)$$

These gradients are then used to update the parameters in the opposite direction of the gradient:

$$\theta = \theta - learningrate * \Delta \theta \quad (2.3)$$

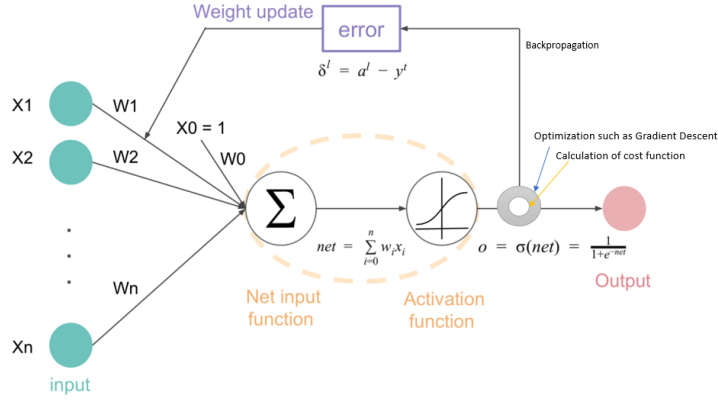


Figure 2.5: Gradient descent and weights update in Backpropagation algorithm [19]

2.3.4 Dropout

Dropout is a regularization technique used in deep learning models to prevent overfitting. It works by randomly dropping out (setting to zero) some of the neurons in the neural network during training. This forces the network to learn more robust features and prevents it from relying too much on any one feature.

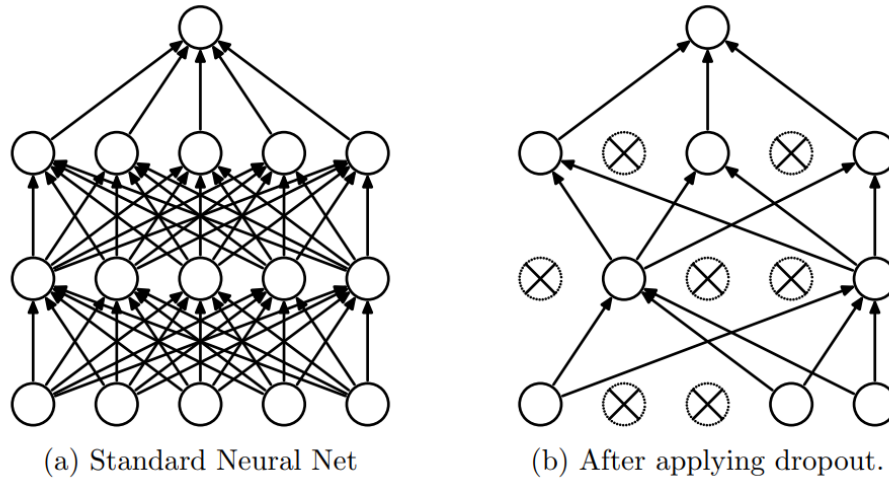


Figure 2.6: Dropout Regularization[20]

2.3.5 Batch normalization

Batch normalization is a technique used to normalize the inputs of each layer. It helps in improving the stability and speed of training by reducing the internal covariate shift,

which refers to the change in the distribution of the network's internal activations during training.

2.3.6 Hyperparameters

Hyperparameters are configuration settings that are set prior to the training of a machine learning model. These settings control various aspects of the training process and significantly impact the model's performance and behavior.

Batch Size

Batch size refers to the number of training examples utilized in one iteration of the model's training. It determines how many samples are processed before updating the model's weights. A larger batch size may lead to faster training but can require more memory, while a smaller batch size may result in slower training but potentially better convergence.

Optimizer

The optimizer is responsible for updating the model's weights during the training process based on the computed gradients. It determines the specific algorithm used to adjust the weights, aiming to minimize the model's loss function. Popular optimizers include Adam, Stochastic Gradient Descent (SGD), and RMSprop.

Learning Rate

The learning rate determines the step size at which the optimizer adjusts the model's weights during training. It controls the speed at which the model learns and converges. A higher learning rate can result in faster initial learning, but it may also cause overshooting or instability. On the other hand, a lower learning rate may lead to slower convergence, but it can provide more accurate results.

Epochs

An epoch represents a complete pass through the entire training dataset during model training. It signifies the number of times the model has seen and learned from all the training samples. Training for more epochs allows the model to refine its predictions and improve

its performance. However, training for too many epochs can lead to overfitting, where the model becomes overly specialized to the training data and performs poorly on unseen data.

These hyperparameters play a crucial role in determining the effectiveness and efficiency of the model training process, and careful selection and tuning of these values are essential for achieving optimal performance.

2.3.7 Activation Functions

An activation function is a key component in artificial neural networks as it determines whether a neuron should be activated or not based on its input. This function plays a crucial role in helping neural networks learn complex patterns in data by determining which neurons get activated and influence the next layer. Similar to how neurons in the human brain activate each other, the activation function takes the output from one neuron and converts it into an input for the next neuron. Its primary purpose is to introduce nonlinearity into the neural network, as without an activation function, a neural network would function solely as a linear regression model.

The activation of a neuron is obtained by applying an activation function to the weighted sum of its inputs, including the bias term as follows:

$$A = G(W_1X_1 + W_2X_2 + W_3X_3 + \dots + W_nX_n + B) \quad (2.4)$$

A: Output of a neuron

G: Activation function

W: Weights

X_i: Features

B: Bias

The choice of activation function impacts the neural network's performance and training process, and common examples include sigmoid, tanh, and ReLU, each having different properties and use cases.

Sigmoid

The sigmoid activation function is a function that squashes the input into an S-shaped curve as shown in Figure, producing outputs between 0 and 1. Therefore, it is used for

models that require a binary classification task. It is defined as

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}} \quad (2.5)$$

Sigmoid activation tends to perform well in shallow neural networks that have only a few hidden layers. However, in deep neural networks, it may become saturated, causing gradients to vanish, which can hinder learning and performance.

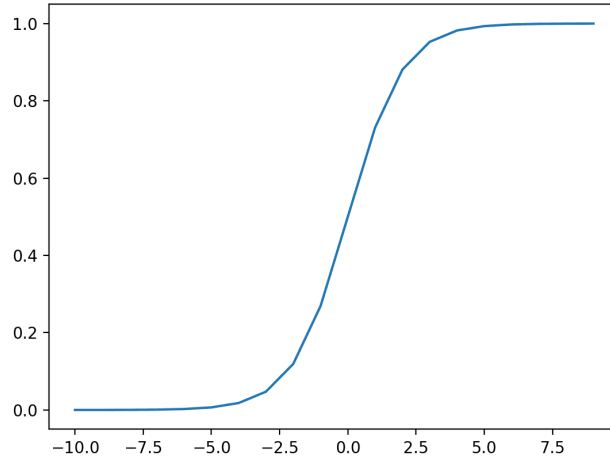


Figure 2.7: Sigmoid activation function [21]

Softmax

Softmax is a generalization of sigmoid for multi-class classification problems. It performs a softmax operation that normalizes the input vector into a probability vector, where each value represents the confidence score or probability of the corresponding class. The predicted class is determined by selecting the class with the highest confidence score. It is commonly employed as the final activation function in the neural network, specifically at the last layer. Softmax is expressed mathematically by

$$\text{Softmax}(\mathbf{x}) = \frac{e^{x_i}}{\sum_{j=1}^K e^{x_j}} \quad (2.6)$$

$\mathbf{x}$ = input vector

$$e^{x_i} = \exp(x_i)$$

K = number of classes in the multi-class classifier

$$e^{x_j} = \exp(x_j)$$

ReLU

ReLU, short for Rectified Linear Unit, is a highly popular activation function in neural networks. It introduces nonlinearity by returning 0 if the input is negative, and the input itself if positive. It is defined as

$$f(\mathbf{x}) = \max(0, x) \quad (2.7)$$

ReLU offers several advantages: as its gradient is either 0 or 1, it enables faster learning compared to sigmoid activations and avoids saturation. It also reduces the chances of vanishing or exploding gradients. Moreover, it strikes a balance between computational efficiency and biological realism, as it outperforms sigmoid in terms of speed.

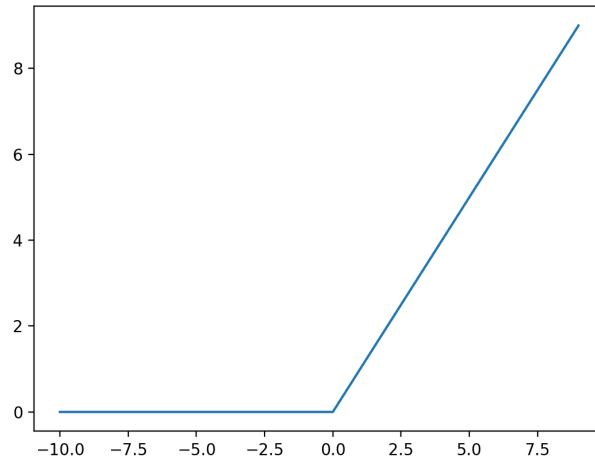


Figure 2.8: ReLU activation function [21]

2.4 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are deep learning models adept at computer vision. By mimicking the visual cortex [22], CNNs utilize convolution operations to detect visual patterns, enabling them to capture both the Spatial and Temporal dependencies in an image through relevant filters. This architectural approach improves the fitting to image datasets by reducing the number of parameters and facilitating weight reusability. Consequently, the network gains a better understanding of the sophistication of the image. With multiple convolution layers constructing a hierarchy of features tuned to different complexities, CNNs excel in computer vision tasks such as image classification. Moreover, ConvNets play a crucial role in reducing images to a more processable form while preserving critical features necessary for accurate predictions. This feature retention is particularly significant when designing scalable architectures capable of handling massive datasets.

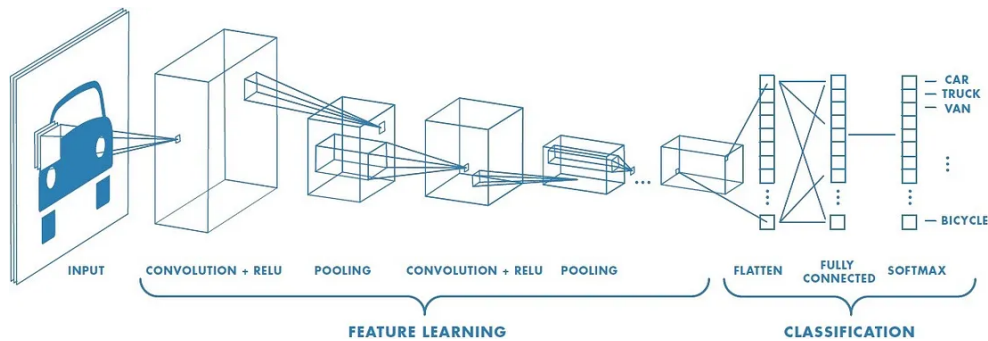


Figure 2.9: A representative structure of CNN [22]

CNNs consist of a stack of alternating convolution layers and pooling layers with a number of fully connected layers.

2.4.1 Convolutional Layer

A convolution layer plays a crucial role in extracting features from an input image. It involves convolving the image with a smaller matrix called a kernel or filter. The kernel slides across the image, computing the dot product between the kernel and the image at each position. The stride length determines the sliding distance. When dealing with colored images, the number of channels in the kernel must match the number of channels in the image. Multiple kernels can be used to extract multiple features, with their sizes kept consistent. The resulting convolved features are stacked together to form an output with channels corresponding to the number of filters used. To introduce non-linearity, an

activation function such as ReLU or Tanh is applied to the convolved features. The output, also known as a feature map, represents the result of this process. This explanation is summarized in Figure 2.10.

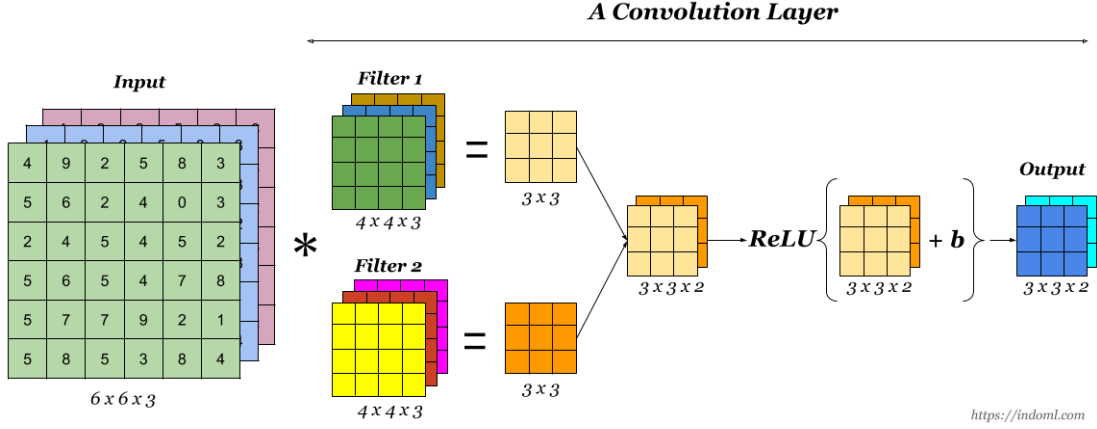


Figure 2.10: Convolution operation on a $6 \times 6 \times 3$ image matrix with a $4 \times 4 \times 3$ KernelR [23]

2.4.2 Pooling Layer

The pooling layer is utilized in convolutional neural networks to reduce the size of the input image and enhance the robustness of detected features. This layer, often placed after a convolutional layer, aims to accelerate computation. Pooling involves using a kernel and stride. In Figure 2.11 a 2×2 filter is used for pooling the 6×6 input image with a stride of 2. Two common types of pooling are max pooling and average pooling. Max pooling selects the maximum value within each patch of a feature map to create a condensed map, while average pooling selects the average value within each patch to form the reduced map.

Max Pooling acts as a noise suppressant by discarding noisy activations and reducing dimensionality, while Average Pooling solely reduces dimensionality without noise suppression. Consequently, Max Pooling outperforms Average Pooling in terms of noise reduction and performance.

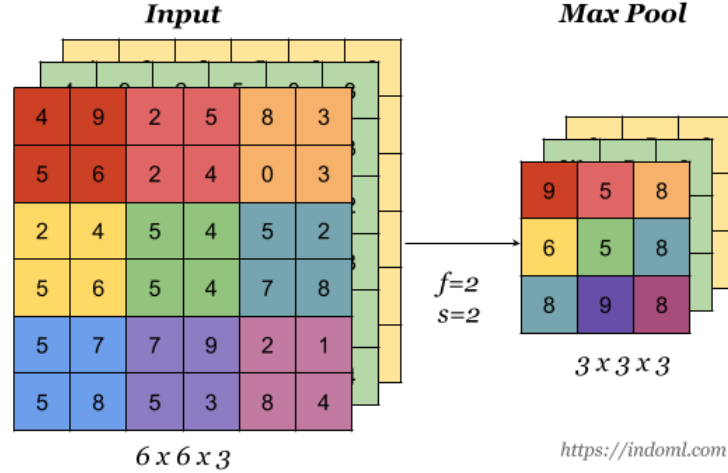


Figure 2.11: Max-pooling layer operation [23]

2.4.3 Fully Connected Layer

The fully connected layer serves as the final component of a convolutional neural network. It operates on the feature map generated by the preceding pooling layer, which is flattened into a vector format. This vector is then passed through the fully connected layer, enabling it to capture intricate image features. Each neuron within the layer represents the probability of a specific feature being present in the image. Consequently, the output of this layer is a one-dimensional feature vector, aligning with the desired number of classes defined by the network's objective. An example is shown in Figure 2.9.

2.5 Related Work

There are two techniques involved in diagnosing diabetic retinopathy: detection and grading. Detection is conducted through binary classification, distinguishing between diabetic retinopathy (DR) and a normal retina. On the other hand, grading of diabetic retinopathy involves identifying and annotating the affected areas, along with categorizing the severity of the infection as mild, moderate, or severe. Figure 2.12 provides an overview of these distinct types of DR studies. In this section, we will conduct a literature review of a selected group of studies, with a focus on deep-learning-based methods, which have proven to be the most effective compared to other machine-learning-based or traditional techniques. The findings of these studies will be summarized in Table 2.2.

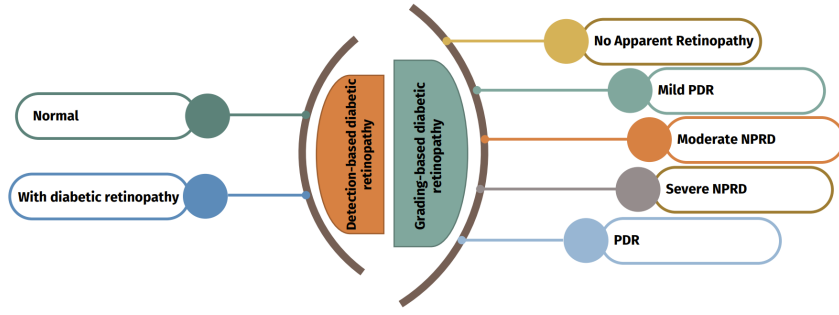


Figure 2.12: Types of diabetic retinopathy studies [24]

The study [25] conducted a comprehensive review of deep learning techniques for detecting and classifying diabetic retinopathy. Various methods including CNNs, UNet, GANs, transfer learning, and ensemble models were analyzed in detail. The review focused on the application of these techniques to fundus images from datasets such as EyePACS, APTOS, Messidor, and DDR. Preprocessing techniques were also discussed, along with the identification of models that demonstrated promising results in real clinical settings. Instead of using basic convolutional neural networks, some researchers have opted for transfer learning or feature extraction using pre-trained models, known as backbones, as illustrated in Figure 2.13. For instance, in [26] HaiQuan et al. employed InceptionV3 for identifying the five stages of diabetic retinopathy. They preprocessed and augmented the EyePACS dataset for training and evaluation, achieving a classification accuracy of 80% and a kappa score of 0.64. Another approach proposed by M. Vijayan and V. S in [27] utilized EfficientNet-B0 as the backbone for a regression-based model in diabetic retinopathy classification. This model achieved an overall multi-class classification accuracy of 85.5% and a kappa score of 0.921 on the APTOS and DDR datasets. In [28], the authors utilized the EfficientNet-B3 backbone with ImageNet weights for training, initializing the fully connected layers using HE initialization, resulting in a Kappa score of 0.935.

DenseNet-121 was employed as a transfer learning-based method for diabetic retinopathy detection in [29]. The developed classifier achieved a significant training accuracy of 96.3% and a validation accuracy of 94.9%, along with a quadratic weighted kappa of 0.88. Furthermore, in [30], transfer learning using VGG, AlexNet, Inception, GoogleNet, DenseNet, and ResNet was utilized for feature extraction, followed by classification using SVM. The proposed method was tested on 77 retinal images from base 12 and 70 retinal images from base 13 of the Messidor database, achieving the highest accuracy values of 95.83% and 95.24% for base 12 and base 13, respectively.

In [31], an ensemble model consisting of three CNN models was employed for classifying diabetic retinopathy (DR). Feature extraction was performed using ResNet-50 and VGG-16, and stack generalization was utilized. The proposed stacked model achieved an overall test accuracy of 97.92% for binary classification and 87.45% for multi-class classification. [32] investigated four factors including network architectures, preprocessing, class imbalance, and fine-tuning, using AlexNet, ResNet-50, and VGG-16. The study reported the best accuracy of 79%. EfficientNet-B7 was employed in [33] to classify diabetic retinopathy into referable/vision-threatening DR, achieving an AUC of 0.984 for RDR and 0.990 for vision-threatening DR on the EyePACS dataset. Similar performance was observed on the APTOS 2019 dataset, with an AUC of 0.966 and 0.998 for referable and vision-threatening DR, respectively. Sai Venkatesh et al. [34] used VGG19, ResNet50, ResNet101, and EfficientNet B0 to B6 architectures to classify the severity of diabetic retinopathy. They achieved accuracies of 73% with VGG and ResNet architectures and 83% with EfficientNetB3. The ensemble EfficientNetB3 model yielded the best result with an accuracy of 93% and a kappa score of 87%. Another study by Shalini et al. [35] proposed two deep learning models, namely CNN and hyperparameter-optimized CNN using Grid Search Cross-Validation (GSCV), for classifying diabetic retinopathy into five different classes. The CNN model achieved an accuracy of 77%, while the hyperparameter-optimized CNN model achieved a higher accuracy of 89%. To improve the classification results, the authors employed classical preprocessing and augmentation techniques.

In a focused study on binary classification for diabetic retinopathy (DR) between normal (class 0) and proliferative DR (class 1), the authors [36] utilized transfer learning CNN models including MobileNetV2, DenseNet, InceptionV3, and VGG16. Retinal images from three different datasets, namely EyePACS, APTOS, and ODIR, were combined for testing. Among the models, DenseNet121 performed the best with an accuracy of 98.97% on the combined image set. This finding highlighted the effectiveness of DenseNet121 and combined datasets for accurately detecting DR.

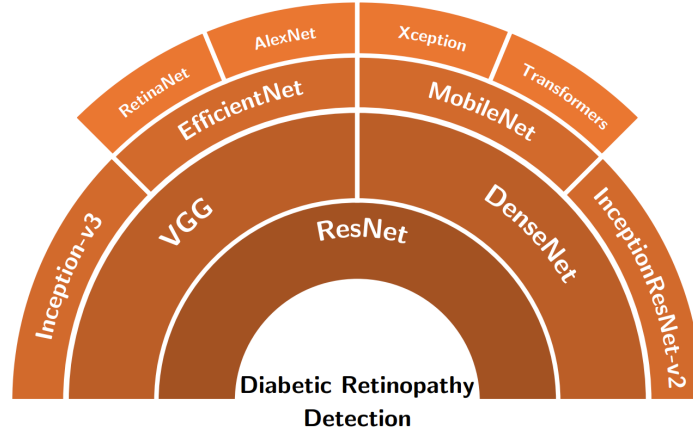


Figure 2.13: Backbones used for diabetic retinopathy detection studies [24]

Despite the advancements in previous studies on diabetic retinopathy, there are some limitations that need to be addressed. These include the use of small datasets, focusing on binary classification and lacking detection of the mild DR stage. Additionally, effectively integrating the complex inter-relationships among different severity levels into the models has posed a significant challenge. Moreover, many studies have primarily reported high accuracy rates on internal datasets, lacking robust validation on external datasets and real-world clinical settings. Lastly, the absence of sensitivity and specificity metrics in some studies undermines the comprehensive evaluation of classification model performance.

Table 2.2: Diabetic Retinopathy detection and grading studies

Study	Dataset	Classification strategy	Architecture	Metric	Value
HaiQuan et al. 2018 [26]	EyePACS	Multi-class (5-class)	InceptionV3	Accuracy QWK	80% 0.64
M. Vijayan and V. S 2023 [27]	APTOS & DDR	Regression based model (5-class)	Regression EfficientNetB0	Accuracy	86.2% 84.8%
Z. Zhang 2020 [28]	APTOS & DDR	Regression-based model (5-class)	EfficientNetB3	QWK	0.935
M. S. Hossen et al. 2020 [29]	APTOS	Multi-class (5-class)	DenseNet-121	Accuracy QWK	94.9% 0.88
D. U. Qomariah et al. 2019 [30]	Messidor Base 12 Base 13	Binary-class (Normal & Severe NPDR)	ResNet50 VGG19 InceptionV3	Accuracy	95% 95.83% 95.24%
H. Kaushik et al. 2021 [31]	EyePACS	Binary-class Multi-class	Stacked Generalisation CNN	Accuracy	97.92% 87.45%
C. Lian et al. 2018 [32]	EyePACS	Multi-class (5-class)	VGG-16	Accuracy	79.04%

Study	Dataset	Classification strategy	Architecture	Metric	Value
M. Chetoui and M. A. Akhloufi 2020 [33]	EyePACS APTOS	Binary-class (RDR & VTDR)	EfficientNetB7	AUC	0.984/0.990 0.966/0.998
Sai Venkatesh et al. 2023 [34]	Kaggle dataset	Multi-class (5-class)	Vgg16 ResNet101 EfficientNetB3 Ensemble- EfficientNetB3	Accuracy QWK	73% 73% 85% 93% 0.87
Shalini et al. 2023 [35]	Private	Multi-class (5-class)	CNN Grid-Search CNN	Accuracy	77% 89%
A. M. Murtawa et al. 2023 [36]	EyePACS, APTOS, ODIR	Binary-class (0 & 4)	DenseNet121	Accuracy	98.97%

2.6 Conclusion

The chapter provided a comprehensive introduction to the fundamental concepts and components of machine learning and deep learning. The discussion on training neural networks, activation functions, and essential network layers has laid a strong foundation for the subsequent chapters focused on applying deep learning techniques to diabetic retinopathy classification. The knowledge gained from this chapter will serve as a crucial framework for the practical implementation and evaluation of deep learning models for DR detection and grading.

Chapter 3

Approach and Implementation

3.1 Introduction

This chapter presents the approach and implementation details for our project. We outline the workflow, discuss the tools and technologies used, describe the datasets employed, cover pre-processing and data augmentation techniques, and introduce evaluation metrics. Additionally, we delve into the concept of transfer learning, specifically focusing on the EfficientNetB1, InceptionV3, Xception, and MobileNetV1 architectures.

3.2 Workflow

Figure 3.1 illustrates the pipeline employed in this work. The image processing workflow begins with loading the image using the appropriate path, followed by preprocessing steps involving filtering, enhancement, and circle cropping. The image then undergoes augmentation to generate multiple variations, which are then used for training with transfer learning models based on CNNs. The ultimate outcome of this process is the detection or grading of diabetic retinopathy (DR).

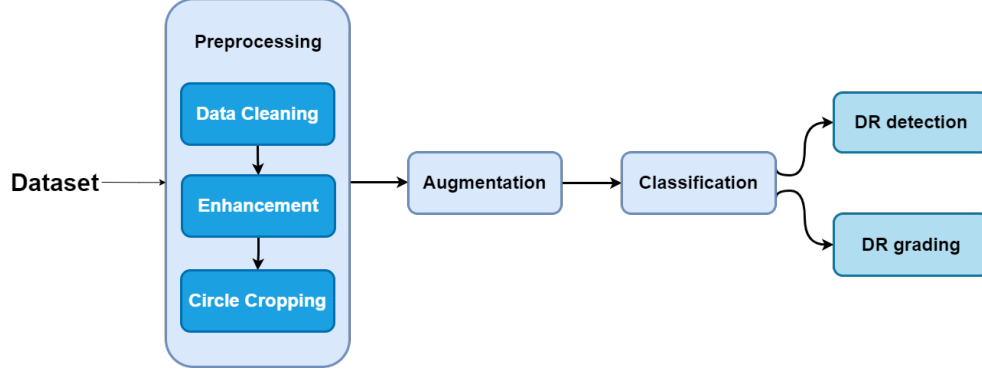


Figure 3.1: General block diagram of the proposed work

3.3 Tools

3.3.1 Python

Python is a widely used programming language and a powerful tool for data manipulation, analysis, and model development. It provides a robust set of libraries and frameworks that make it a popular choice for various data science and machine learning projects.

3.3.2 TensorFlow

TensorFlow, an open-source project by Google, is a powerful framework for executing numerical computations in machine learning. It offers APIs for multiple languages and handles data manipulation through tensors and data flow graphs.

3.3.3 Keras

Keras, integrated with TensorFlow in 2017 [37], provides a user-friendly interface for building deep learning models. It offers preconfigured APIs and predesigned modules that allow developers to create customized neural network models easily. Keras supports external libraries like scikit-learn, enabling the use of various algorithms and tools for data preprocessing and machine learning tasks.

3.3.4 Scikit-learn

Scikit-learn is a popular open-source Python library that provides efficient tools for data analysis and mining. It supports a range of supervised and unsupervised learning algorithms, including classification, regression, clustering, and dimensionality reduction. With a user-friendly interface, Scikit-learn seamlessly integrates with NumPy and pandas, while also providing visualization capabilities through Matplotlib integration.

3.3.5 OpenCV

OpenCV is a versatile open-source library for computer vision and machine learning. It offers numerous algorithms for tasks like recognition, tracking, 3D modeling, stitching, and text detection. Supporting Deep Neural Networks with frameworks like PyTorch and TensorFlow, OpenCV runs efficiently on various hardware architectures and is available in multiple languages. With over 2500 algorithms and comprehensive documentation, OpenCV is a powerful and flexible library for vision applications.

3.3.6 Kaggle

Kaggle is an online platform for data science competitions, collaboration, and education. It enables users to discover and publish datasets, collaborate with other professionals in the field, and engage in data science competitions. Kaggle offers a web-based data science environment where users can explore, analyze, and build models using Jupyter Notebooks. Additionally, it provides access to a virtual machine with a free GPU accelerator, empowering users to leverage high-performance computing for their ML tasks. We used it for both the development and testing phases.

3.4 Datasets

3.4.1 EyePACS

Kaggle EyePACS is the largest and most used public dataset for DR classification, it contains 35,126 high-resolution fundus images of the retina captured from both eyes, obtained using diverse devices and under various imaging conditions across multiple care sites in California. However, due to such variability, both the data (e.g. artifacts, blurring, focusing

and exposure problems) and the ground truth labels exhibit noise. This dataset was made available by the EyePACS platform for the Diabetic Retinopathy Detection competition [38], supported by the California Healthcare Foundation. Clinicians graded these fundus photographs on a scale of 0 to 4, reflecting the severity of Diabetic Retinopathy (DR).

3.4.2 APTOS

Kaggle APTOS 2019 Challenge [39] was gathered by the Aravind Eye Hospital in rural areas of India. The objective was to develop powerful tools that could automatically diagnose Diabetic Retinopathy and enhance the hospital’s capability to identify potential patients. This dataset is the third largest in size and comprises 3662 images. However, Similar to the Kaggle EyePACS the dataset has a significant class imbalance, particularly in the Severe NPDR class, which consists of only 193 images.

3.4.3 DDR

The DDR [25] dataset is relatively new and hasn’t been extensively used. It comprises 12,522 images that were collected from 147 hospitals in China and annotated by multiple professionals. To address the issue of poor-quality images, a sixth grade was included to categorize them separately. However, there is a significant class imbalance between the healthy/moderate DR classes and the remaining classes, namely mild, severe, and proliferative DR. In terms of diabetic retinopathy (DR) lesions, 757 images in the dataset were annotated at a pixel level for lesion segmentation purposes.

Table 3.1 presents the five classes of DR along with their corresponding proportions.

Table 3.1: Distribution of EyePACS, APTOS, and DDR datasets

Class	EyePACS	APTOS	DDR
0 : No DR	25810	1805	6266
1 : Mild	2443	370	630
2 : Moderate	5292	999	4477
3 : Severe	873	193	236
4 : Proliferative DR	708	295	913
Total number of images	35126	3662	12522

3.5 Pre-processing

The process of capturing fundus images involves using different hardware devices and dealing with various environmental conditions, which introduces noise into the final image. In order to reduce such heterogeneity, which ultimately affects the performance of the classification model, as well as to enhance the visibility of fine details within the images, pre-processing is commonly employed in the majority of the reviewed studies. In the preprocessing phase, several steps were taken to prepare the fundus image dataset for further analysis. The following steps were performed:

3.5.1 Data Refinement

Bright and Dark Images Images come with different lighting conditions some images are very bright while others are very dark and impossible to visualize as shown in figure 3.2.

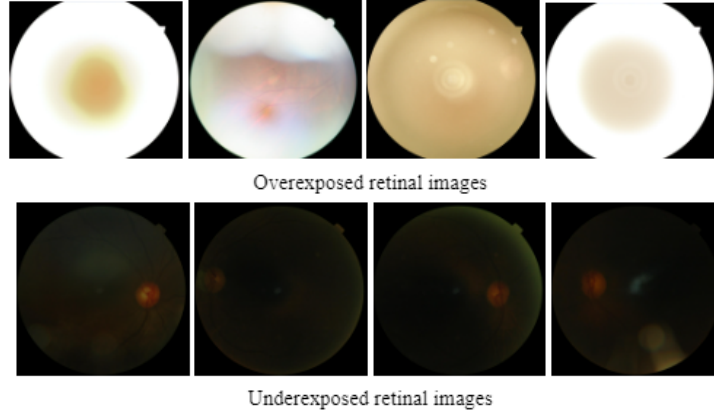


Figure 3.2: Examples of overexposed and underexposed images from the dataset

To address the issue of images with poor visibility of retinal features, a function was implemented to filter out fundus images that appeared either too bright or too dark, lacking clear features of the retina. The function determines the brightness level of each image based on its L channel values in the LAB color space and drops those whose L channel exceeds the brightness threshold or below the darkness threshold. The threshold values were tuned after a series of trials. Elimination of destroyed images Images with significant flaws, such as eye lashes' shadow, partially captured content, or other forms of destruction as shown in figure 3.3, were deemed unfit for the dataset. These images were removed from the dataframe to maintain data quality and integrity.

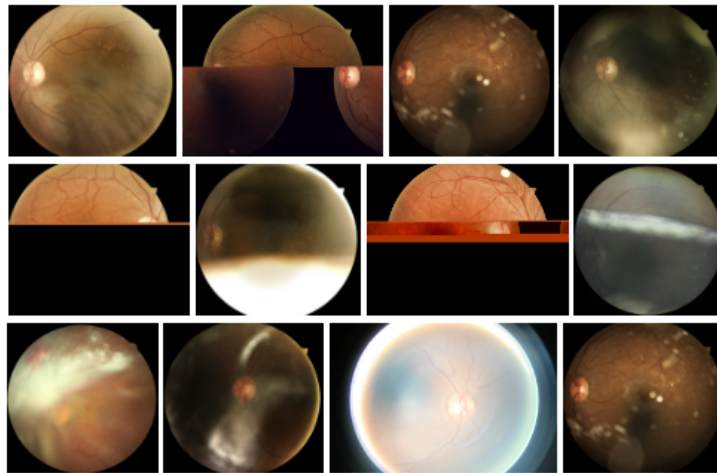


Figure 3.3: Unfit images in the dataset

A total of 10517 images were removed from the dataset after filtering out overexposed, underexposed, and destroyed images.

3.5.2 Image Enhancement

To ensure that the fundus images contained clear and enhanced retinal features the following preprocessing steps were performed: Normalization First, the images are normalized to account for variations in illumination, contrast, and color balance.

- **CLAHE equalization**

The RGB (R-Red, B-Blue, G-Green) channels of the fundus images were extracted and the Contrast Limited Adaptive Histogram Equalization (CLAHE) [40] technique was applied to equalize the histogram of each channel, improving the contrast and enhancing the visibility of blood vessels and lesions in the retina.

- **Gaussian blur for noise removal**

Gaussian blur, a smoothing technique, was employed to reduce noise in the images. this step helped to eliminate unwanted artifacts and to enhance the overall quality of the fundus images.

- **Morphological cleaning**

This technique was applied to remove isolated pixels that may introduce noise or disturbances in the image. This process involved identifying and eliminating isolated pixels or small clusters, resulting in a cleaner and more accurate representation of blood vessels.

- **Resizing**

Finally the images were resized to a standard size of 224x244 dimensions.

Figure 3.4 shows the changes made on two images after enhancement.

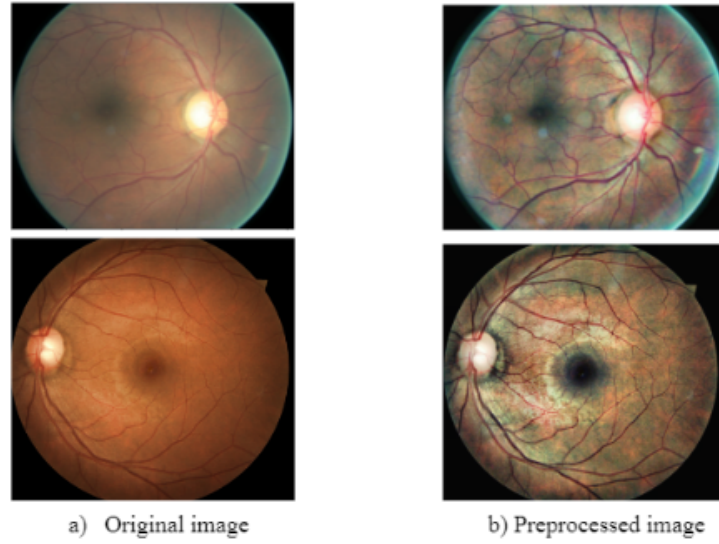
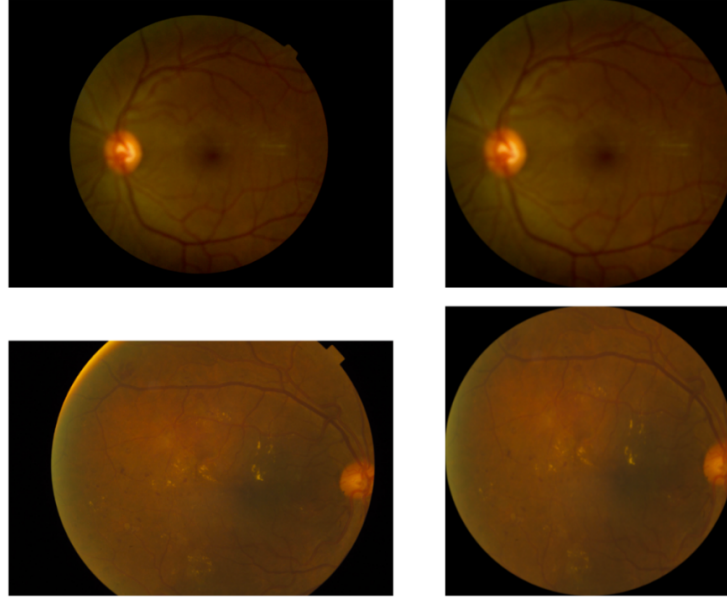


Figure 3.4: Retinal image preprocessing

3.5.3 Circular Cropping

The images were center-cropped to remove black pixels and irrelevant information, ensuring that only the fundus image portion remains for processing. This helps enhance feature extraction and mitigate potential outliers or noise present in the image. The function utilizes two techniques: `crop_image_from_gray` and `circle_crop`. The first creates a binary mask based on a tolerance value to remove black borders and irrelevant information from the image. And the second builds upon the first by creating a circular crop around the center using a filled circle mask, further enhancing the image.



a) Original

b) Circle Cropped

Figure 3.5: Circle cropped retinal images

3.6 Data augmentation

As can be seen in Figure 3.6, we face a significant class imbalance as we have a considerable number of healthy retinal images compared to images with DR, in this case, the task of classifying diabetic retinopathy becomes challenging.



Figure 3.6: Bar graph of the final dataset distribution

3.6.1 Traditional Augmentation Techniques

To overcome the issue of an imbalanced dataset, we implemented several commonly used techniques for augmenting DR dataset using ImageDataGenerator from Keras. These techniques involve modifying the existing images to create new images that can be added to the dataset such as rotation, flipping, cropping, and scaling.

Table 3.2: Parameter settings for augmentation

Augmentation Method	Setting
Horizontal flip	True
Rotation range	$[-90^\circ, 90^\circ]$
Width & Height shift	$[-0.2, 0.2]$
Zoom	$[-20\%, 20\%]$

3.6.2 Augmentation using Morphological Filters

Morphological filters are image processing techniques based on mathematical operations [41] that analyze and modify the shape, structure, or texture of objects within an image. The most basic morphological filters include dilation, erosion, opening, and closing.

Dilation: Dilation expands or thickens the boundaries of objects in an image by replacing each pixel with the maximum value within its neighborhood. It is useful for filling gaps or joining nearby regions.

Erosion: Erosion shrinks or erodes the boundaries of objects in an image by replacing each pixel with the minimum value within its neighborhood. It helps in removing small details or isolating objects.

Opening: Opening is a combination of erosion followed by dilation. It is effective in removing small objects or noise while preserving the overall structure of larger objects.

Closing: Closing is a combination of dilation followed by erosion. It helps in closing small holes or gaps within objects while preserving the overall shape.

Figure 3.7 illustrates the application of four operations with a structuring element of radius 3 on a single image.

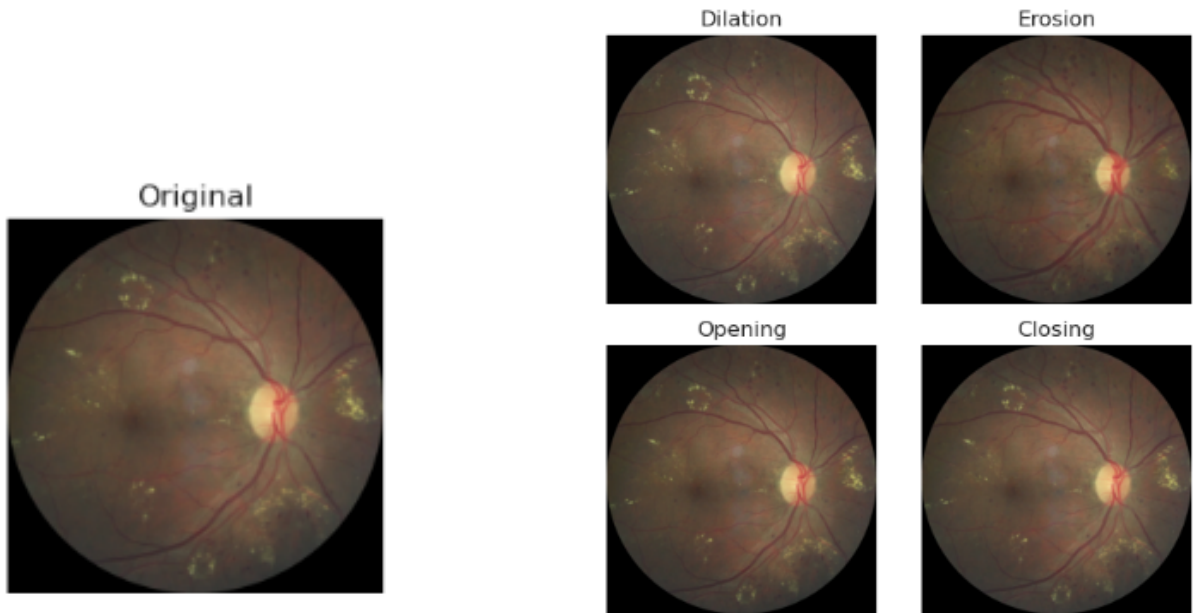


Figure 3.7: Morphological operation on a retinal image

3.7 Evaluation metrics

Evaluation metrics play a crucial role in assessing the effectiveness of neural networks, and their selection depends on the specific application. Among the commonly used metrics for classifier performance, four key indicators are essential:

True Positive (TP): Correctly identifying the presence of a class or characteristic.

True Negative (TN): Correctly identifying the absence of a class or characteristic.

False Positive (FP): Incorrectly identifying the presence of a class or characteristic.

False Negative (FN): Incorrectly identifying the absence of a class or characteristic.

These indicators serve as the foundation for all classification evaluation metrics.

3.7.1 Confusion matrix

A confusion matrix is a valuable tool for evaluating the performance of a classification model. It provides a tabular representation of the model's correct and incorrect predictions, categorized by the actual and predicted classes. This matrix is particularly useful when dealing with imbalanced datasets where class distribution varies. By analyzing the patterns of errors, it becomes possible to identify areas where the model may be struggling, enabling targeted improvements to enhance overall performance.

		Actual Class		
		A	B	C
Predicted Class	A	TP_A	E_{BA}	E_{CA}
	B	E_{AB}	TP_B	E_{CB}
	C	E_{AC}	E_{BC}	TP_C

Figure 3.8: An illustrative example of the confusion matrix for a multi-class classification test

TP_A = elements of class A that are correctly classified as class A
 E_{AB} = elements of class A that are incorrectly classified as class B
 E_{AC} = elements of class A that are incorrectly classified as class C
 E_{BA} = elements of class B that are incorrectly classified in class A
 TP_B = elements of class B that are correctly classified in class B
 E_{BC} = elements of class B that are incorrectly classified in class C
 E_{CA} = elements of class C that are incorrectly classified in class A
 E_{CB} = elements of class C that are incorrectly classified in class B
 TP_C = elements of class C that are correctly classified in class C

3.7.2 Accuracy

Accuracy is a measure of how often a classification model correctly predicts the class of an input sample, expressed as a percentage of the total number of samples. It is calculated using the following formula:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (3.1)$$

3.7.3 Precision

In classification, precision is the proportion of true positive predictions out of the total predicted positive instances. It measures the accuracy of positive predictions, indicating how well the model identifies the relevant instances. A high precision value indicates that the model makes fewer false positive predictions.

$$Precision = \frac{TP}{TP + FP} \quad (3.2)$$

3.7.4 Sensitivity (Recall)

Sensitivity is a measure of a classification model's ability to correctly identify positive cases, also known as true positives, for each class. It is calculated by dividing the number of true positives by the total number of actual positives.

$$Sensitivity = \frac{TP}{TP + FN} \quad (3.3)$$

3.7.5 F1-score

The F1-score is a harmonic mean of precision and recall and is used as a measure of overall classification performance. It ranges from 0 to 1, with higher values indicating better performance. The F1-score is useful when the class distribution is imbalanced. Its primary application is to compare the performance of two classifiers.

$$F1 - score = \frac{Precision * Sensitivity}{Precision + Sensitivity} \quad (3.4)$$

3.7.6 Weighted average

The weighted average is an evaluation metric that calculates the average performance of a model across multiple classes, taking into account the class distribution by assigning higher weights to classes with larger populations. It provides a comprehensive measure of overall model performance, accounting for imbalanced class distributions.

3.8 Transfer learning

Transfer learning is a powerful technique in which knowledge gained from one model is applied to another related task. In our project, we have employed transfer learning using the EfficientNetB1, InceptionV3, Xception, and MobileNetV2 architectures, utilizing pre-trained models trained on large datasets such as ImageNet. The EfficientNet, InceptionV3, Xception, and MobileNetV2 architectures are renowned for their effectiveness in diabetic retinopathy classification [24]. We have specifically chosen these models due to their capability to achieve remarkable performance while having a relatively smaller number of parameters. This choice is well-suited for our dataset, which comprises a large number of high-resolution images.

3.8.1 EfficientNetB1

EfficientNetB1 is a state-of-the-art CNN architecture that was introduced in 2019 by Tan et al. [42] from Google Research. It is part of the EfficientNet family, which aims to achieve better trade-offs between model size and accuracy compared to existing CNN models. The EfficientNetB1 architecture utilizes a building block called the inverted residual block, which is illustrated in Figure 3.10 and referred to as MBConv in Figure 3.9. This block

incorporates the squeeze and excitation technique; the input feature maps are first expanded depth-wise using 1×1 convolutions. This expansion helps to increase the richness and dimensionality of the features. Then, a combination of a 3×3 depth-wise convolution and a point-wise convolution is applied to reduce the number of channels in the output feature map. This reduction helps to maintain computational efficiency [43].

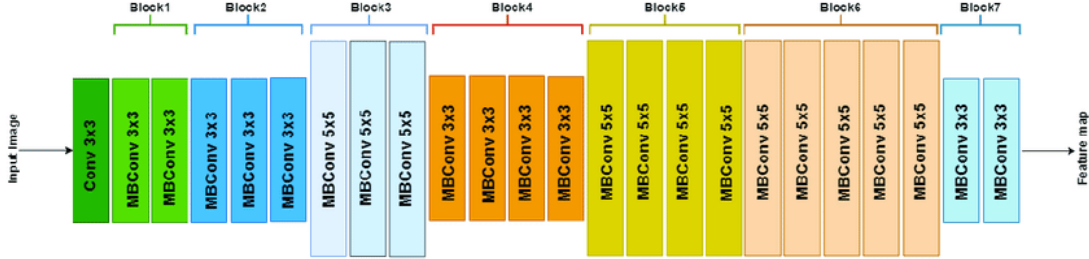


Figure 3.9: Architecture of EfficientNetB1 with MBConv as basic building blocks [44]

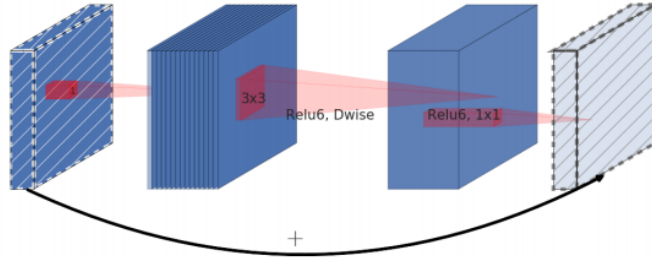


Figure 3.10: Structure of an inverted residual block (MBconv) [44]

EfficientNet uses a compound scaling method that scales the network width, depth, and resolution in a systematic way to achieve better accuracy and efficiency. Trained on large-scale datasets, including the ImageNet dataset, which contains a vast number of images from 1000 different classes, EfficientNetB1 has been successfully applied to a wide range of real-world applications and shown remarkable results in tasks such as image classification, object detection, and semantic segmentation.

3.8.2 InceptionV3

InceptionV3 is a convolutional neural network architecture that was introduced by Christian Szegedy et al. [44] from Google Research in 2015. It is part of the Inception family

of models, which are known for their effectiveness in image classification and object recognition tasks. It utilizes factorized convolutions and aggressive regularization to reduce the number of parameters and prevent overfitting. The architecture consists of Inception modules that perform parallel convolutions with different filter sizes and pooling operations, allowing the network to capture features at various scales. InceptionV3 also incorporates auxiliary classifiers at intermediate stages to provide additional supervision during training as shown in Figure 3.11. With its powerful features and efficient design, InceptionV3 has achieved state-of-the-art results on various computer vision tasks.

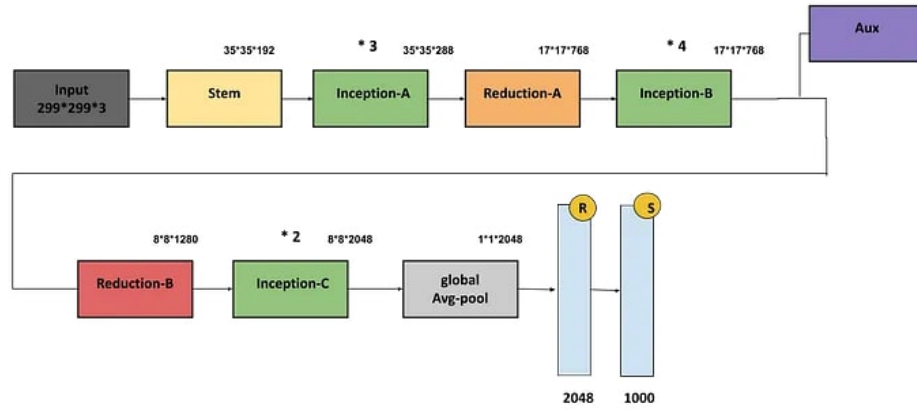


Figure 3.11: InceptionV3 architecture [45]

3.8.3 Xception

The Xception architecture also known as the "extreme" version of an Inception module is based entirely on depthwise separable convolution layers. This architecture is inspired by the Inception architecture, which uses Inception modules to extract features from images. However, Xception takes this a step further by completely decoupling the mapping of cross-channel correlations and spatial correlations in the feature maps of CNNs [46]. This means that the Xception architecture can use model parameters more efficiently than the Inception architecture, resulting in better performance on image classification tasks. The Xception architecture has 36 convolutional layers forming the feature extraction base of the network. Figure 3.12 shows the modified depthwise separable convolution used as the Inception module in Xception.

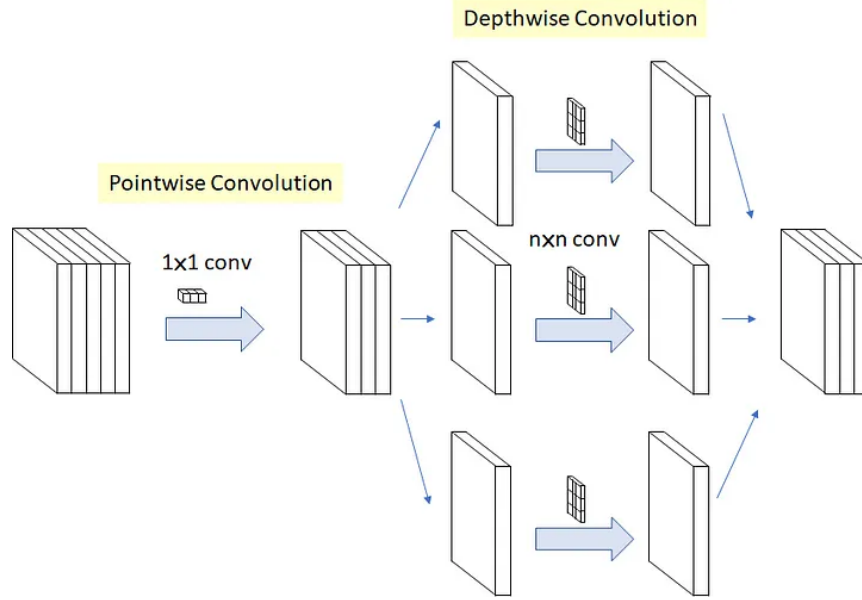


Figure 3.12: Modified depth-wise separable convolution for Xception architecture [47]

3.8.4 MobileNetV2

MobileNetV2 is a lightweight CNN architecture designed for mobile and embedded vision applications, improving upon the original MobileNet. It utilizes depthwise separable convolutions to maintain accuracy while reducing model size. Linear bottlenecks and lightweight depthwise convolutions minimize computation cost. The architecture includes an improved width multiplier for channel pruning, residual connections for faster convergence, and offers two models: a high-accuracy baseline and a smaller model for low-powered devices [48, 49].

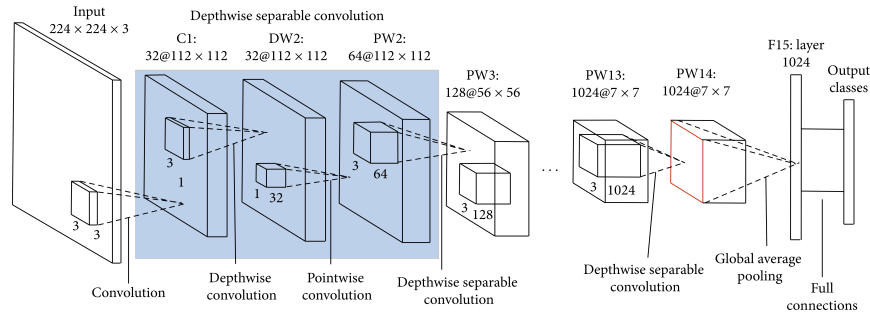


Figure 3.13: MobileNet architecture [49]

3.9 Conclusion

Chapter 3 provides an overview of the methodology and tools used in the project. It highlights the datasets used, the preprocessing and augmentation techniques, evaluation metrics, and the used transfer learning models. Overall the chapter establishes the foundation for the next chapter's results and discussion laying the groundwork for the development and evaluation of a deep learning model for diabetic retinopathy.

Chapter 4

Results and Discussion

4.1 Introduction

This chapter represents the outcomes and discussion of our study of diabetic retinopathy classification using deep learning techniques. The primary objective is to conduct an extensive evaluation of various factors related to employing deep convolutional neural networks for DR classification such as CNN architectures, image preprocessing techniques, handling class imbalance and overfitting, and hyperparameter tuning.

The evaluation of these factors was conducted in five distinct phases. Phase 1 involved assessing the performance of VGG and AlexNet architectures, while Phase 2 incorporated the utilization of EfficientNetB1, image preprocessing techniques, class imbalance handling through augmentation, and hyperparameter fine-tuning. In Phase 3, we extended our analysis by applying transfer learning architectures such as Xception, InceptionV3, and MobileNet. Additionally, Phase 4 involved the exploration of morphological filters in an effort to mitigate overfitting. Finally, Phase 5 focused on addressing binary classification tasks. The experimental setup and evaluation metrics used in this study were described in Section 3.8 and Section 3.7, respectively.

4.2 Phase 1: Initial Training

As a first step, the EyePACS dataset (before refinement and augmentation) was divided into 70% for training and 30% for validation and trained using the AlexNet and VGG16 architectures. However, the results obtained were unsatisfactory.

		Confusion Matrix				
Actual	No_DR	3914	0	0	0	0
	Mild	333	0	0	0	0
	Moderate	785	0	0	0	0
	Severe	115	0	0	0	0
	Proliferate_DR	119	0	0	0	0
		No_DR	Mild	Moderate	Severe	Proliferate_DR
		Predicted				

Figure 4.1: Confusion matrix of the first training on AlexNet and VGG16 models

As shown in Figure 4.1, the model recognized only class 0 (No_DR) which happens to be the largest class in the dataset, and failed to classify the remaining 4 classes. This was evident from the poor performance depicted in the loss and accuracy graphs for the training and validation sets (Figure 4.2 and Figure 4.3).



Figure 4.2: Loss and accuracy graphs of AlexNet model.

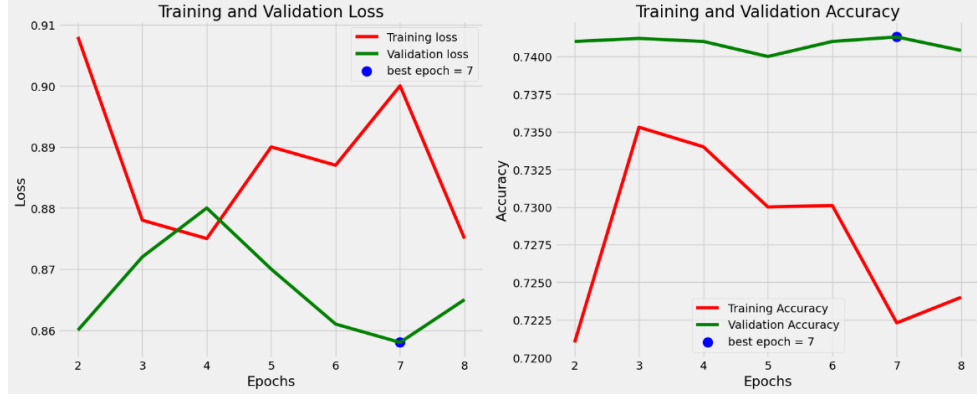


Figure 4.3: Loss and validation graphs of the VGG16 model.

To address the limitations encountered in the initial training, a careful selection of datasets, transfer learning models, and hyperparameters was conducted.

We employed the pre-trained architectures from the Keras library as the foundation of our models. Additionally, we extended these architectures by incorporating batch normalization, a dense layer with 256 units, a dropout layer with a rate of 0.4 to address overfitting, another dense layer with 128 units and a dropout rate of 0.5, and finally softmax activation as the output layer. These modifications aimed to enhance the model’s capacity to extract meaningful features and improve its overall performance.

To determine the optimal hyperparameters for training our models, an iterative process of trial-and-error was conducted. We carefully evaluated various combinations of hyperparameter values to identify the most effective ones. After thorough experimentation, we selected the hyperparameters shown in Table 4.1

Table 4.1: Selected hyperparameter values

Hyperparameters	Selected value
Batch size	32
Learning rate	0.001
Loss	categorical_crossentropy
Optimizer	Adam

Additionally, we used a custom Keras callback class that dynamically adjusts the learning rate during training. It monitors either the training accuracy or validation loss based on a threshold value (0.93) to determine which metric to track. If the monitored metric does not improve within the specified patience (1 epoch), the learning rate is reduced by a factor of (0.3).

Initially, the number of epochs is set to 30. However, it can be observed that the actual number of epochs varies across different experiments due to the utilization of an early stopping mechanism and our custom callback class. This adaptive approach allowed us to automatically stop the training process when the model’s performance no longer showed significant improvement, preventing unnecessary computational resources from being expended. As a result, the actual number of epochs varied depending on the convergence of the model during training.

The train-validation-test-split method is chosen with a ratio of 70:20:10 for model selection.

4.3 Phase 2: Experiments on EfficientNetB1

In this phase the model was trained using EfficientNetB1 architecture, and the experiments were carried out gradually. Firstly, we trained the model on the raw EyePACS dataset. Subsequently, we applied the data-cleaning process mentioned in section 3.5.1 to remove low-quality images. In the third experiment, we combined the EyePACS dataset with the APTOS and DDR datasets to further enhance the training process.

To provide an overview of the model used in each experiment, the corresponding model summary is presented. This summary outlines the architecture and structure of the model, showcasing its layers, parameters, and output dimensions. The architecture of the EfficientNetB1 model is provided in Figure 4.4

Model: "sequential_2"

Layer (type)	Output Shape	Param #
efficientnetb1 (Functional)	(None, 1280)	6575239
batch_normalization_3 (Batch Normalization)	(None, 1280)	5120
dense_5 (Dense)	(None, 256)	327936
dropout_6 (Dropout)	(None, 256)	0
dense_6 (Dense)	(None, 128)	32896
dropout_7 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 5)	645

Total params: 6,941,836
 Trainable params: 6,877,221
 Non-trainable params: 64,615

Figure 4.4: EfficientNetB1 model summary

4.3.1 EfficientNetB1 Experiment I

We initiated the training process using the raw EyePACS dataset. The subsequent performance on training and validation can be observed in Figure 4.5.

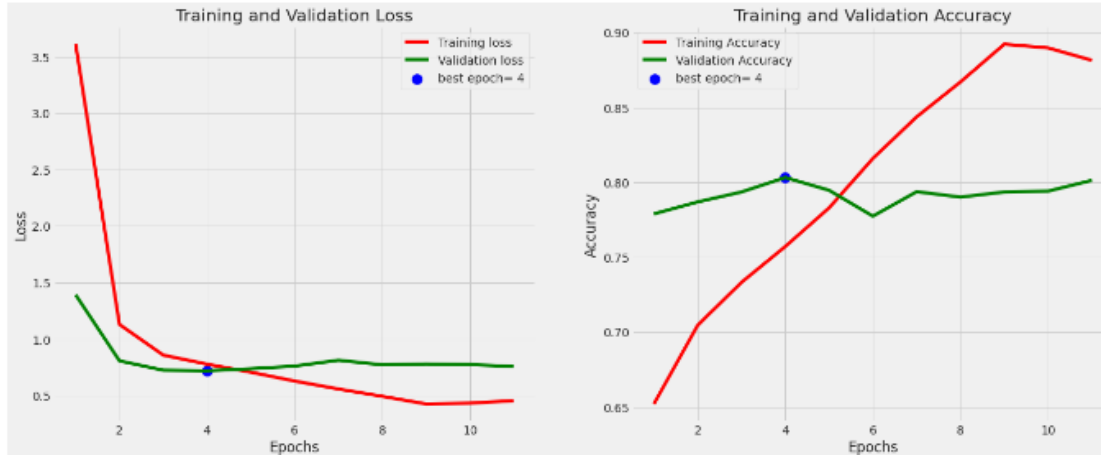


Figure 4.5: Loss and validation graphs of the EfficientNetB1 model during experiment I

The observed overfitting curve suggests that the model has learned to perform exceptionally well on the training data, but its performance on the validation set did not improve over

time. This indicates the issue of overfitting, where the model has memorized the training examples instead of learning the underlying patterns.

The classification report is represented in Table 4.2, and the associated confusion matrix is visualized in Figure 4.6.

Table 4.2: Classification report on EfficientNetB1’s first experiment

Classes	Precision	Recall	F1-score	Total accuracy
No_DR	0.83	0.96	0.89	
Mild	0.00	0.00	0.00	
Moderate	0.58	0.43	0.50	78.77%
Severe	0.35	0.14	0.20	
Proliferate_DR	0.54	0.62	0.58	
Weighted average	0.71	0.79	0.74	

		Confusion Matrix				
Actual	No_DR	3714	1	116	0	22
	Mild	349	0	29	0	1
	Moderate	395	1	340	24	25
	Severe	18	0	75	18	16
	Proliferate_DR	11	1	26	9	76
		No_DR	Mild	Moderate	Severe	Proliferate_DR
		Predicted				

Figure 4.6: Confusion matrix of EfficientNetB1 during experiment I

As can be seen in the classification report and confusion matrix, the model indicates strong performance in accurately identifying healthy cases. However, it shows moderate performance in classifying Proliferate_DR and Moderate classes. The model exhibits challenges in correctly identifying mild cases, as all metrics were reported as 0.00, this suggests that the model struggled to differentiate mild cases from other categories.

Overall the total accuracy of the model on the EyePACS dataset was reported as 78.77%. These results highlight the challenges faced by the model in accurately classifying mild and severe cases, indicating areas of improvement.

4.3.2 EfficientNetB1 Experiment II

In the second experiment, we refined the dataset by eliminating underexposed, overexposed, and destroyed images. The objective was to assess the impact of this data-cleaning process on the performance of the classifier. We trained the model and plotted the training and validation loss and accuracy. The graphs are shown in Figure 4.7

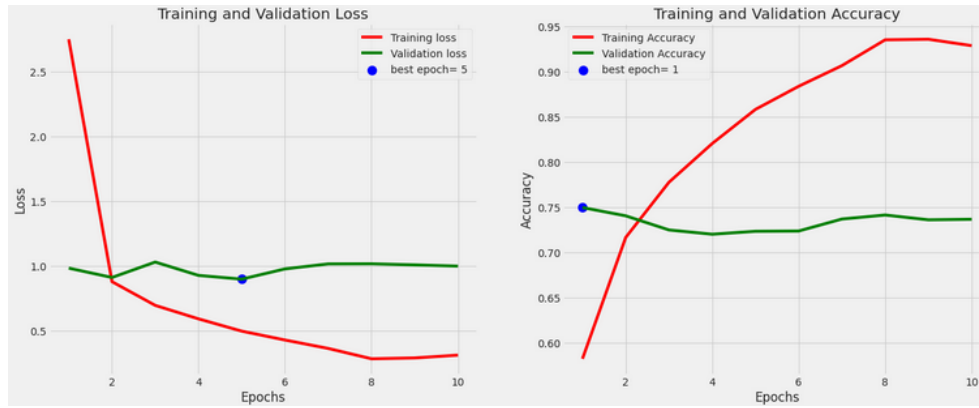


Figure 4.7: Loss and validation graphs of EfficientNetB1 model during experiment II

The graph shows an increasing trend in training accuracy, indicating that the model's performance improves as it trains on the data. However, the validation accuracy indicates that the model's performance is reaching a plateau, where further training does not significantly improve the results. The classification report and confusion matrix are provided in Table 4.3 and Figure 4.8, respectively.

Table 4.3: Classification report of EfficientNetB1 experiment II

Classes	Precision	Recall	F1-score	Total accuracy
No_DR	0.80	0.88	0.84	
Mild	0.23	0.16	0.19	
Moderate	0.56	0.52	0.54	72.12%
Severe	0.66	0.21	0.32	
Proliferate_DR	0.77	0.66	0.71	
Weighted average	0.70	0.72	0.71	

Actual	No_DR	2005	124	139	1	1
	Mild	230	49	30	0	1
	Moderate	246	35	321	8	5
	Severe	11	3	62	23	10
	Proliferate_DR	3	0	23	3	57
		No_DR	Mild	Moderate	Severe	Proliferate_DR
		Predicted				

Figure 4.8: Confusion matrix of EfficientNetB1 during experiment II

After the data refinement process, notable improvements can be observed in the classification results. The No_DR class has remained consistently high, indicating the model's ability to accurately identify healthy images. Additionally, there is a noticeable increase in the precision, recall, and F1-score for the Moderate and Severe classes, suggesting an

improvement in classifying images with moderate and severe diabetic retinopathy.

While the results for the Mild class are still not satisfactory, it is worth noting that there has been an increase in detection compared to the previous results. This indicates that the model is becoming better at recognizing mild cases of diabetic retinopathy.

Overall, the data-cleaning process has positively impacted the classifier’s performance by enhancing the metrics for certain classes. Although there is still room for improvement, these results demonstrate the potential of data-cleaning techniques in improving the accuracy and reliability of the classifier.

4.3.3 EfficientNetB1 Experiment III

To enhance the performance of our classifier, we made the decision to augment the initial dataset (EyePACS) with additional datasets (APTOS and DDR). This choice was motivated by two primary factors. Firstly, we observed that our classifier was struggling to detect the Mild class, which resulted in lower accuracy and limited the overall performance of the model. By incorporating additional datasets, we aimed to provide more diverse and representative samples of the mild class, thereby enabling the classifier to better learn and differentiate this specific category.

Secondly, the EyePACS dataset has a significantly lower number of images in classes Mild, Severe, and Proliferate_DR compared to the remaining classes, with class 0 being 36 times bigger than class 4. This class imbalance poses challenges for the classifier, as it leads to biased predictions and a focus on the majority class. By introducing supplementary datasets, we aimed to address this issue by providing a better distribution of samples across the different classes. This approach helps the model to learn effectively and make better accurate predictions across all classes, including the ones with limited representation in the original dataset.

The training and validation performance is shown in Figure 4.9.

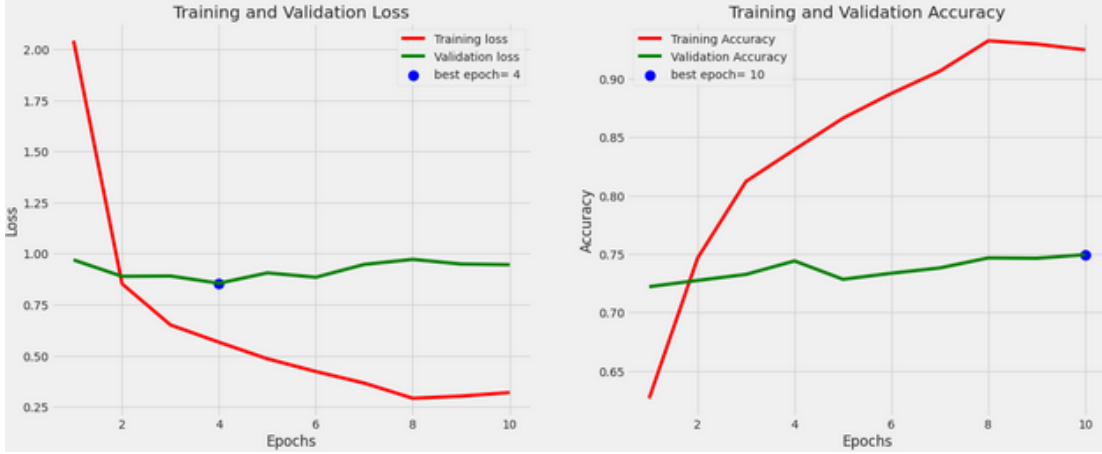


Figure 4.9: Loss and validation graphs of EfficientNetB1 model during experiment III

The model still overfits the train data although dropout regularization was used. The classification report of this model when evaluated is given in Table 4.4 and the confusion matrix in Figure 4.10.

Table 4.4: Classification report of EfficientNetB1 experiment III

Classes	Precision	Recall	F1-score	Total accuracy
No_DR	0.88	0.90	0.89	80.15%
Mild	0.19	0.13	0.19	
Moderate	0.65	0.65	0.65	
Severe	0.56	0.45	0.50	
Proliferate_DR	0.79	0.71	0.74	
Weighted average	0.81	0.78	0.80	

Actual	No_DR	2878	267	178	2	3
	Mild	161	65	51	1	2
	Moderate	213	64	542	41	27
	Severe	9	2	41	52	11
	Proliferate_DR	6	0	34	11	124
		No_DR	Mild	Moderate	Severe	Proliferate_DR
		Predicted				

Figure 4.10: Confusion matrix of EfficientNetB1 during experiment III

The results of the third experiment on EfficientNetB1 show good overall performance and improvement, with high precision, recall, and F1-score for the majority of classes. The model achieves a total accuracy of 80.15%, demonstrating its effectiveness in classifying diabetic retinopathy severity levels. The weighted average scores indicate a balanced performance across all classes, highlighting the model’s capability in handling imbalanced datasets.

4.4 Phase 3

In this phase, we evaluate the performance of additional state-of-the-art transfer learning architectures, namely Xception, InceptionV3, and MobileNet, for diabetic retinopathy. We thoroughly analyze the performance of these architectures and assess their effectiveness in our study.

4.4.1 Experiment on InceptionV3

The second model was trained using the InceptionV3 architecture following the various experiments conducted on the EfficientNetB1. The experiment that showcased the most

promising results was selected as a benchmark along with its corresponding configuration. The architecture of the entire model is provided in Figure 4.11

Model: "sequential"

Layer (type)	Output Shape	Param #
inception_v3 (Functional)	(None, 2048)	21802784
batch_normalization_94 (Batch Normalization)	(None, 2048)	8192
dense (Dense)	(None, 256)	524544
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 128)	32896
dropout_1 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 5)	645

=====
Total params: 22,369,061
Trainable params: 22,330,533
Non-trainable params: 38,528
=====

Figure 4.11: InceptionV3 model summary

We trained the model and plotted the training and validation loss and accuracy. The graphs are shown in Figure 4.12

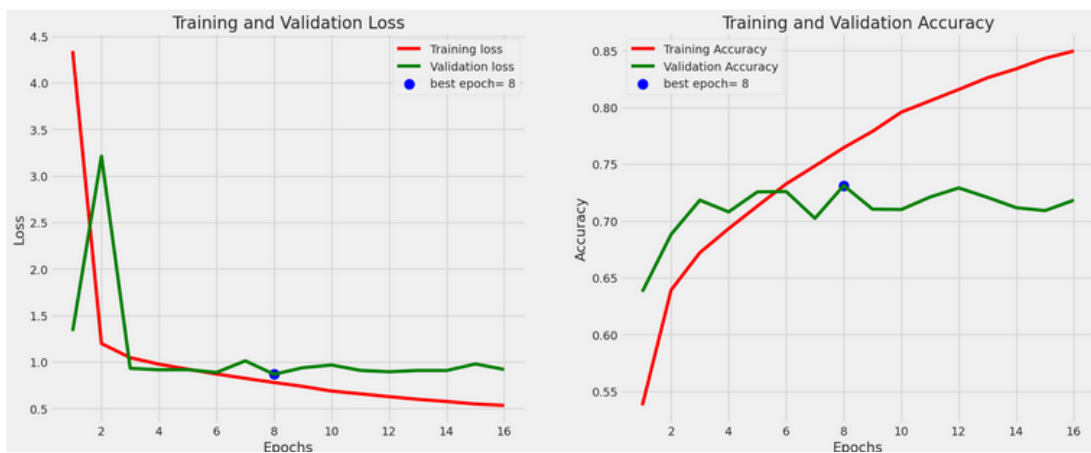


Figure 4.12: Loss and validation graphs of InceptionV3 model

The validation loss and accuracy values fluctuate slightly after a certain number of epochs.

This could be an indication of the model's performance reaching a plateau, where further training does not significantly improve the results. The classification report and confusion matrix of this model when evaluated on the test set are given in Table 4.5 and Figure 4.13, respectively.

Table 4.5: Classification report of InceptionV3 model

Classes	Precision	Recall	F1-score	Total accuracy
No_DR	0.85	0.97	0.91	81.65%
Mild	0.23	0.04	0.07	
Moderate	0.70	0.58	0.64	
Severe	0.48	0.27	0.35	
Proliferate_DR	0.73	0.45	0.56	
Weighted average	0.78	0.81	0.79	

Actual	No_DR	3279	18	69	0	1
	Mild	228	12	27	0	1
	Moderate	314	19	515	17	17
	Severe	10	2	60	31	11
	Proliferate_DR	12	2	69	16	82
		No_DR	Mild	Moderate	Severe	Proliferate_DR
		Predicted				

Figure 4.13: Confusion matrix of InceptionV3 model

The InceptionV3 model demonstrates overall good performance in classifying diabetic retinopathy. It achieves high performance for the No_DR class, indicating accurate identification of healthy retinas. However, it struggles with the detection of the Mild class which remains a challenge. Despite this, the model shows promising results for the Moderate and Proliferate_DR classes, with respectable F1-scores.

4.4.2 Experiment on Xception

The next transfer learning model is Xception. Similarly, we followed the same steps as the previous architectures. The architecture of this model is provided in Figure 4.14

Model: "sequential_1"

Layer (type)	Output Shape	Param #
xception (Functional)	(None, 2048)	20861480
batch_normalization_99 (Batch Normalization)	(None, 2048)	8192
dense_3 (Dense)	(None, 256)	524544
dropout_2 (Dropout)	(None, 256)	0
dense_4 (Dense)	(None, 128)	32896
dropout_3 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 5)	645
Total params: 21,427,757		
Trainable params: 21,369,133		
Non-trainable params: 58,624		

Figure 4.14: Xception model summary

This model's obtained accuracy and loss graphs are given in Figure 4.15.

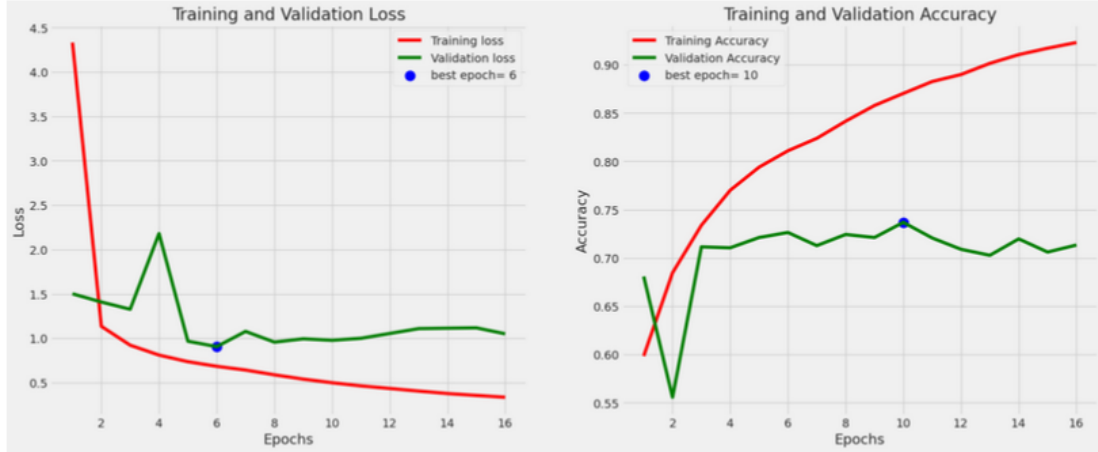


Figure 4.15: Loss and validation graphs of Xception model

From the previous graph, we can notice that there was an overfitting for both accuracy and loss although regularization techniques were used. The different evaluation metrics for each class and their weighted average are provided in Table 4.6 and the confusion matrix is presented in Figure 4.16.

Table 4.6: Classification report on Xception experiment

Classes	Precision	Recall	F1-score	Total accuracy
No_DR	0.86	0.95	0.90	80.28%
Mild	0.17	0.07	0.10	
Moderate	0.66	0.58	0.62	
Severe	0.49	0.30	0.37	
Proliferate_DR	0.78	0.51	0.62	
Weighted average	0.77	0.80	0.78	

Actual	No_DR	3204	58	100	1	4
	Mild	209	18	41	0	0
	Moderate	308	20	513	26	15
	Severe	6	4	62	34	8
	Proliferate_DR	10	4	65	9	93
		No_DR	Mild	Moderate	Severe	Proliferate_DR
		Predicted				

Figure 4.16: Confusion matrix of Xception model

The Xception model performs reasonably well in categorizing diverse stages of DR with some variation in performance across different classes.

4.4.3 Experiment on MobileNetV2

The last transfer learning model is MobileNetV2, the architecture of this model is shown in Figure 4.17.

Model: "sequential_2"

Layer (type)	Output Shape	Param #
mobilenetv2_1.00_224 (Functional)	(None, 1280)	2257984
batch_normalization_100 (Batch Normalization)	(None, 1280)	5120
dense_6 (Dense)	(None, 256)	327936
dropout_4 (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 128)	32896
dropout_5 (Dropout)	(None, 128)	0
dense_8 (Dense)	(None, 5)	645

Total params: 2,624,581
 Trainable params: 2,587,909
 Non-trainable params: 36,672

Figure 4.17: MobileNetV2 model summary

This model's obtained accuracy and loss graphs are given in Figure 4.18:



Figure 4.18: Loss and validation graphs of MobileNetV2 model

The graph reveals major overfitting, despite the implementation of regularization techniques. Additionally, the validation loss and accuracy exhibit marginal fluctuations, indicating limited improvement with further training.

Table 4.7 presents the evaluation metrics for individual classes and their weighted average, while Figure 4.19 illustrates the confusion matrix.

Table 4.7: Classification report on MobileNetV2 experiment

Classes	Precision	Recall	F1-score	Total accuracy
No_DR	0.90	0.84	0.87	
Mild	0.16	0.17	0.16	
Moderate	0.52	0.69	0.59	75.12%
Severe	0.37	0.25	0.30	
Proliferate_DR	0.37	0.25	0.30	
Weighted average	0.77	0.75	0.76	

Actual	No_DR -	2816	177	362	7	5
	Mild -	147	46	70	3	2
	Moderate -	155	59	606	27	35
	Severe -	2	3	68	29	12
	Proliferate_DR -	2	5	56	12	106
		No_DR -	Mild -	Moderate -	Severe -	Proliferate_DR -
		Predicted				

Figure 4.19: Confusion matrix of MobileNetV2 model

The MobileNetV2 model demonstrates a moderate performance in terms of precision, recall, and F1-score across different classes. However, compared to the previous models it achieves

a slightly lower overall accuracy. The model exhibits higher precision and recall for the No_DR class but struggles with lower performance in detecting the Mild, Severe, and Proliferate_DR classes. Overall, it provides a satisfactory weighted average, but further improvements could be explored to enhance its performance on specific classes.

4.5 Phase 4: Applying Morphological Operations

Alongside the challenge of an imbalanced dataset, we faced the issue of overfitting. To tackle this, we sought to enhance our image augmentation strategy by integrating morphological filters, specifically dilation, erosion, opening, and closing with a structuring element of radius 3. These filters were implemented in conjunction with our existing augmentation techniques. Our choice of model for training was InceptionV3, as it demonstrated the highest accuracy in previous experiments. Given our limited training resources, we opted to train the model for only eight epochs, striking a balance between performance and efficiency.

By applying these morphological filters, we aimed to introduce variations to the images in the dataset, thereby improving the performance of the deep neural networks in accurately classifying diabetic retinopathy stages.

Figure 4.20 show the model's obtained loss and accuracy graphs.

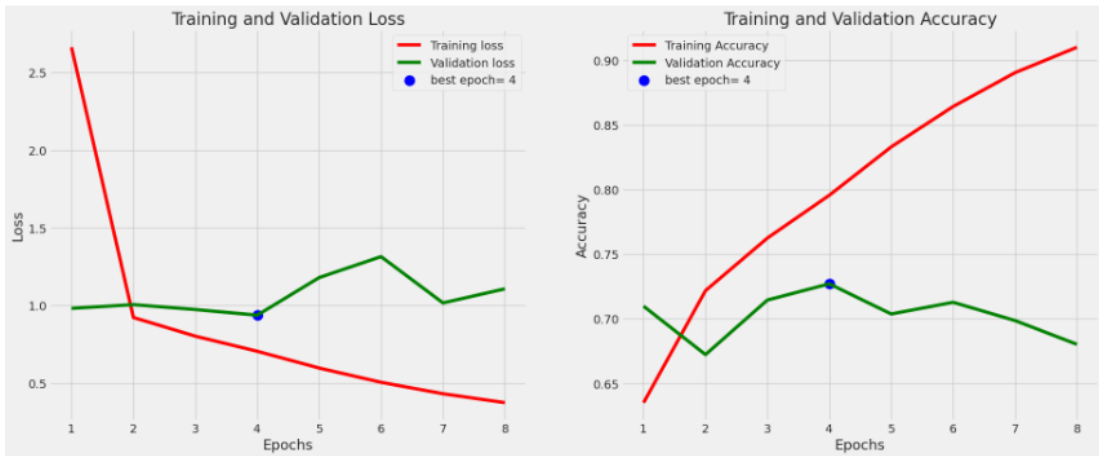


Figure 4.20: Loss and validation graphs of InceptionV3 model with morphological filters augmentation

The graphs clearly demonstrate that the problem of overfitting remains unresolved. However, there is a noticeable enhancement in the model's ability to classify minority classes,

as evidenced by the classification report in Table 4.8 and the comparison of confusion matrices before and after the integration of morphological filters, as illustrated in Figure 4.21, where the left and right panels represent the matrices before and after, respectively.

Table 4.8: Classification report on InceptionV3 using morphological operations

Classes	Precision	Recall	F1-score	Total accuracy
No_DR	0.90	0.85	0.87	75.10%
Mild	0.13	0.19	0.16	
Moderate	0.56	0.65	0.60	
Severe	0.44	0.31	0.36	
Proliferate_DR	0.78	0.48	0.59	
Weighted average	0.78	0.75	0.76	

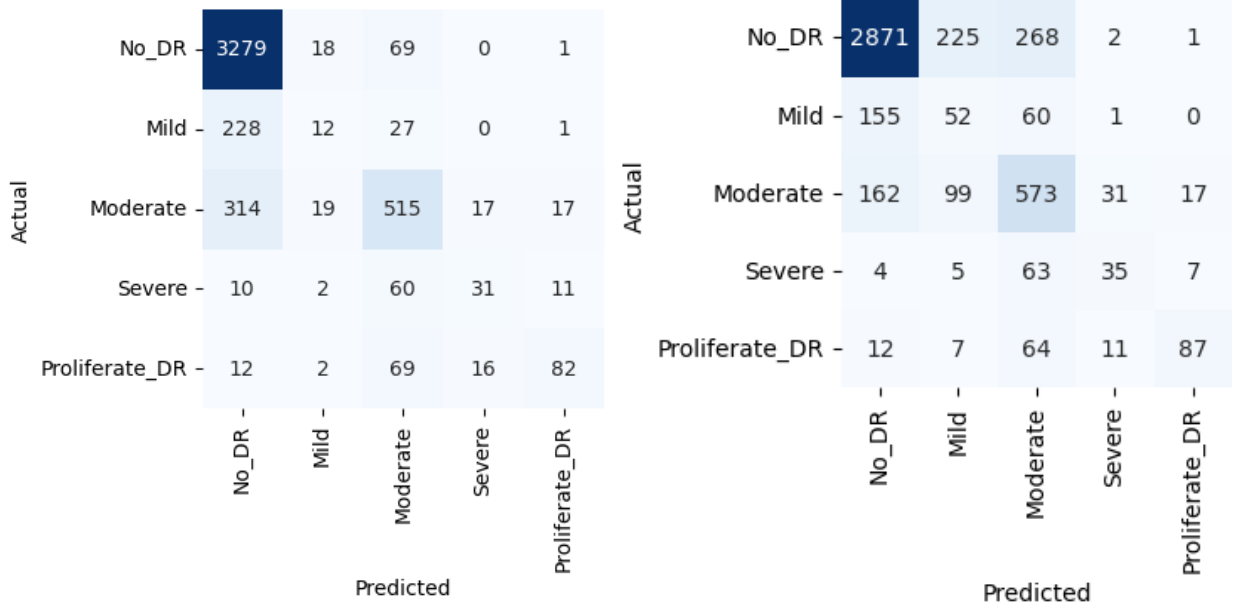


Figure 4.21: Confusion matrices before and after applying morphological operations

4.6 Phase 5: Binary Classification

Grading diabetic retinopathy into its four stages poses significant challenges, particularly in accurately detecting the Mild class and handling data imbalance in the severe classes. To overcome these challenges and gain insights into the performance of our models under simplified conditions, we explored binary classification tasks. By categorizing retinas into two classes, namely No_DR (healthy) and DR (unhealthy) as shown in Figure 4.22, we aimed to evaluate the models' effectiveness in detecting diabetic retinopathy. This approach allowed us to focus on the presence or absence of the disease, specifically addressing the difficulties associated with the subtle symptoms of the Mild class and the limited availability of images in the severe classes.



Figure 4.22: Binary class distribution

We trained the four earlier used models in grading DR. Their confusion matrices are depicted in Figure 4.23.

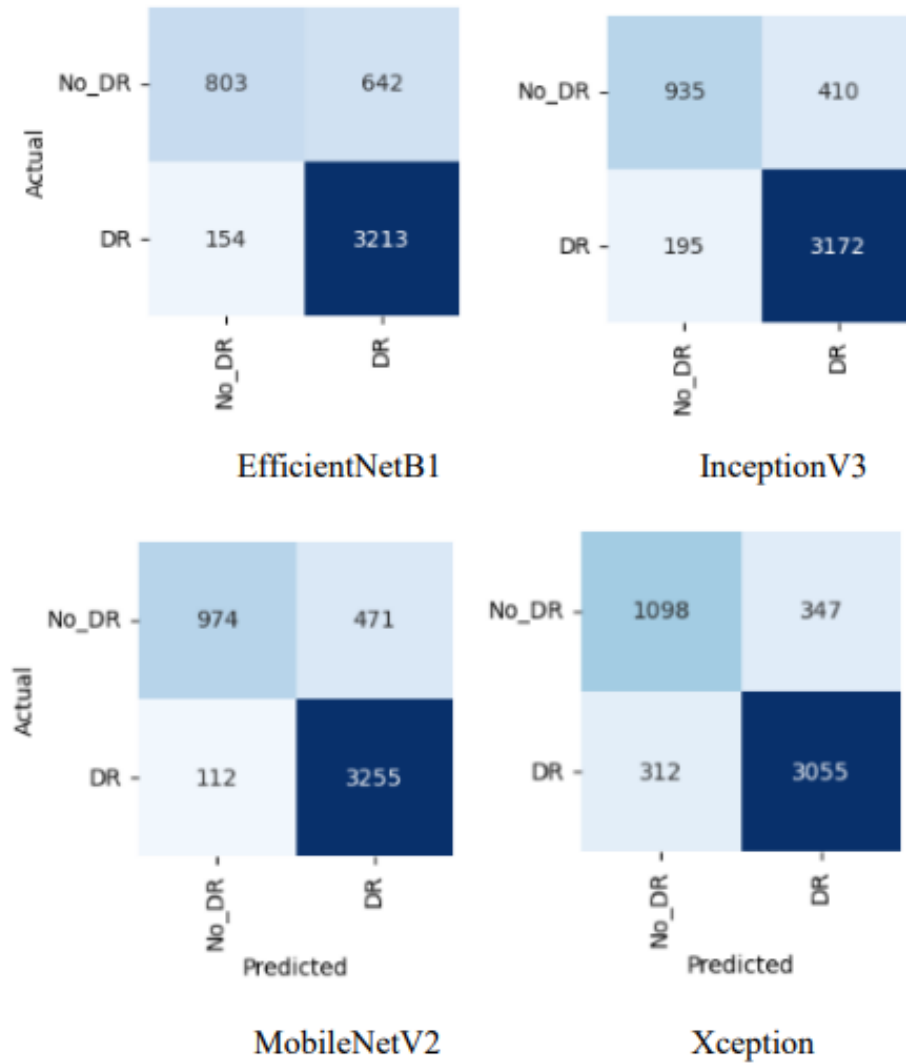


Figure 4.23: Confusion matrices of the four models for binary classification

Despite the comparable performance among the models, MobileNetV2 was the best. Its accuracy and loss graphs are shown in Figure 4.24.



Figure 4.24: Loss and validation graphs of MobileNetV2 model for binary classification

The graphs reveal that the loss remains consistently below 0.5, and there is only a small amount of overfitting observed in the validation accuracy. These findings indicate improved results compared to those obtained in grading experiments.

Table 4.9 displays the evaluation metrics for the No_DR and DR classes across the four models.

Table 4.9: Classification report of the four models for binary classification

Architecture	Classes	Precision	Recall	F1-score	Total accuracy
EfficientNetB1	No_DR	0.83	0.95	0.89	83.46%
	DR	0.84	0.56	0.67	
	Weighted average	0.84	0.83	0.82	
InceptionV3	No_DR	0.89	0.94	0.91	87.43%
	DR	0.84	0.72	0.77	
	Weighted average	0.87	0.87	0.87	
MobileNetV2	No_DR	0.87	0.97	0.92	87.88%
	DR	0.90	0.67	0.77	
	Weighted average	0.88	0.88	0.87	
Xception	No_DR	0.90	0.91	0.90	86.31%
	DR	0.78	0.76	0.77	
	Weighted average	0.86	0.86	0.86	

MobileNetV2 outperformed the other architectures in terms of accuracy and other evaluation metrics, showcasing its effectiveness in classification tasks. Additionally, its relatively small parameter count compared to the other models results in shorter training time and efficient memory usage. This makes MobileNetV2 a practical choice, striking a balance between performance, efficiency, and avoiding overfitting concerns.

4.7 Summary of Results

The overall comparison with some of the related works is depicted in Table 4.10

Table 4.10: Summary of results and overall comparison among experiments

Study	Architecture	Accuracy	Precision	Recall	F1-score
Sai Venkatesh et al [34]	EfficientNetB1	78%	0.69	0.83	0.78
	EfficientNetB3	85%	0.85	0.72	0.84
HaiQuan et al [28]	InceptionV3	80%	0.76	0.80	0.77
H. Kaushik et al [31]	Stacked Generalisation CNN	87.45%	-	-	-
C. Lian et al [32]	VGG	79.04%	-	-	-
Our study	EfficientNetB1	80.15%	0.81	0.77	0.80
	InceptionV3	81.65%	0.78	0.81	0.79
	Xception	80.28%	0.77	0.80	0.78
	MobileNetV2	75.12%	0.77	0.75	0.76

4.8 Discussion

The classification of diabetic retinopathy (DR) into its five stages is a challenging task, particularly in accurately detecting the early stages from retinal images. Previous research efforts have primarily focused on binary or multi-class classification, but they often struggled to achieve accurate results for the early stages. In this study, we conducted experiments to assess the performance of four deep learning models, namely EfficientNetB1, InceptionV3, Xception, and MobileNetV2, for DR classification.

One notable observation from the evaluation reports on the test set is that the classification of the Mild class proved to be the most challenging among the five classes. This can be

attributed to the high similarity between Mild stage images and those labeled as No.DR, as well as the subtle symptoms and features characteristic of this stage. This difficulty likely contributed to the lower performance in recognizing and correctly classifying stage 1 images.

Comparing the performance of the different models, it is evident that InceptionV3 achieved the highest overall accuracy of 81.65%. However, when considering precision, recall, and F1-score for individual classes, the EfficientNetB1 model consistently demonstrated better performance, indicating its superior ability to accurately classify the various severity levels of diabetic retinopathy. Moreover, EfficientNetB1 offers the additional advantage of having fewer parameters compared to InceptionV3, resulting in shorter training times. This makes EfficientNetB1 a more efficient and practical choice for diabetic retinopathy classification tasks, striking a balance between accuracy and training efficiency.

We also encountered the challenge of overfitting during our experiments. Despite implementing various techniques to mitigate this issue, including the use of morphological filters which resulted in improved classification, we were unable to fully resolve the plateau phenomenon. The validation accuracy remained constant without significant improvement over epochs. However, in binary classification tasks, the problem of overfitting was significantly mitigated, shedding light on the substantial impact of class imbalance which was eliminated in binary classification. This insight underscores the importance of further addressing this issue by delving deeper into morphological filters technique by exploring more operations and different structuring elements, and considering additional approaches such as class weighting and specialized algorithms tailored to handle imbalanced datasets.

As a future direction, it would be valuable to extend this work to include the detection and classification of diabetic retinopathy severity with even lower error rates. By leveraging the strengths of the models employed in this study and further refining their capabilities, we can aim for more precise and reliable severity classification, which would have significant implications for the early detection and treatment of diabetic retinopathy.

4.9 Conclusion

In conclusion, this chapter presented the results and discussion of our experiments on diabetic retinopathy (DR) classification using different deep learning models. The experiments revealed that EfficientNetB1 consistently demonstrated strong performance across various evaluation metrics, indicating its effectiveness in accurately classifying different severity levels of DR. These findings highlight the importance of selecting an appropriate model architecture for DR classification, considering both overall accuracy and class-specific performance. Future work should focus on exploring additional approaches to resolve the overfitting issue, refining the models and enhancing the precision in classifying severe levels, aiming for even higher accuracy and reliability in detecting and classifying DR.

General Conclusion

The spread of Diabetic Retinopathy is a growing concern worldwide as it often remains undetected, making early detection crucial to prevent vision loss.

In this work, an experimental evaluation of a Deep Learning-based image classification approach is presented in order to classify the severity of Diabetic Retinopathy from retinal images. The proposed work deploys four transfer learning models (EfficientNetB1, InceptionV3, Xception, and MobileNetV2) trained using a combination of three datasets (EyePACS, APTOS, and DDR) consisting of approximately 31,000 images of healthy and non-healthy eyes, which were screened under different conditions. The project investigates the impact of various factors such as image preprocessing, class imbalance, and hyperparameter tuning on the performance of deep CNNs for DR classification.

The experimental evaluation demonstrates that our system achieves an accuracy of 80.15% and 81.65% with EfficientNetB1 and InceptionV3, respectively. However, the results highlight the limitations of the models in detecting the Mild and Severe classes, underscoring the need for further improvement in future work. Nevertheless, compared to related work, the findings affirm the potential of our AI-based diagnosis system to assist ophthalmologists in diagnosing DR more efficiently and accurately, ultimately aiding in early detection and treatment to prevent blindness.

Looking ahead, future work in this field could involve expanding the dataset to encompass diverse populations, improving the performance of the transfer learning models for challenging cases, and incorporating additional evaluation metrics such as Quadratic Weighted Kappa to comprehensively assess the system's performance. Another area of future work is the introduction of explainability techniques, specifically eXplainable Artificial Intelligence (XAI) to the DR classification process. By incorporating XAI, the system can provide transparent and interpretable explanations for its predictions, enhancing trust and understanding for both healthcare professionals and patients. Additionally, creating

a user-friendly web or mobile app could assist healthcare professionals and patients in detecting and grading diabetic retinopathy. Such an app would integrate deep learning models, allowing users to capture retinal images for analysis and receive real-time feedback on the presence and severity of the disease. By bringing advanced technology to the fingertips of healthcare professionals and patients, the app could revolutionize the way diabetic retinopathy is diagnosed and monitored.

Bibliography

- [1] “Idf diabetes atlas, 9th edition.” [Online]. Available: <https://diabetesatlas.org/atlas/ninth-edition/>
- [2] J. e. a. Krause, “Grader variability and the importance of reference standards for evaluating machine learning models for diabetic retinopathy,” *Ophthalmology*, vol. 125, p. 1264–1272, 2018.
- [3] G. for Developers, “Case study: Tensorflow in medicine - retinal imaging (tensorflow dev summit 2017),” Feb 2017. [Online]. Available: <https://www.youtube.com/watch?v=oOeZ7IgEN4o>
- [4] C. E. W. et al, “Anatomy and physiology of the human eye: effects of mucopolysaccharidoses disease on structure and function - a review,” *Clinical and Experimental Ophthalmology*, vol. 38, no. 22, pp. 2–11, 2010.
- [5] A. M. Helmenstine, “Structure and function of the human eye,” ThoughtCo Available: thoughtco.com/how-the-human-eye-works-4155646, 2021.
- [6] UCL, “The retina and retinal pigment epithelium (rpe),” *UCL Institute of Ophthalmology*, 2020. [Online]. Available: <https://www.ucl.ac.uk/ioo/research/research-labs-and-groups/carr-lab/bestrophinopathies-resource-pages/eye/retina-and-retinal>
- [7] N. Maurya, “Anatomy of retina,” Ph.D. dissertation, 2019. [Online]. Available: <https://medicoapps.org/m-anatomy-of-retina/>
- [8] H. Kolb, “Simple anatomy of the retina,” *Journal of Neurocytology*, vol. 41, no. 6, pp. 349–350, 2012. [Online]. Available: <https://webvision.med.utah.edu/book/part-i-foundations/simple-anatomy-of-the-retina/>
- [9] J. W. Y. et al, “Global prevalence and major risk factors of diabetic retinopathy,” *Diabetes Care*, vol. 35, no. 3, pp. 556–564, 2012.

- [10] D. S. W. T. et al, “Development and validation of a deep learning system for diabetic retinopathy and related eye diseases using retinal images from multiethnic populations with diabetes,” *JAMA*, vol. 318, no. 22, pp. 2211–2223, 2017.
- [11] B. E. Institute, “Diabetic retinopathy honolulu — diabetic eye disease kailua-kona,” 9 2021. [Online]. Available: <https://www.bennetteyeinstitute.com/retina-honolulu/diabetic-retinopathy/>
- [12] V. K. et al, “Towards a new biomarker for diabetic retinopathy: Exploring rbp3 structure and retinoids binding for functional imaging of eyes in vivo,” *International Journal of Molecular Sciences*, vol. 24, no. 5, p. 4408, 2023.
- [13] I. Dev, “Iris fundus photography,” 2023. [Online]. Available: <https://retinalscreenings.com/blog/fundus-photography-and-the-importance-of-quality-retinal-images/>
- [14] S. Pandian, “Understand machine learning and its end-to-end process,” 2023. [Online]. Available: <https://www.analyticsvidhya.com/blog/2020/12/understand-machine-learning-and-its-end-to-end-process/>
- [15] V. G. et al, “Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs,” *JAMA*, vol. 316, no. 22, pp. 2402–2410, 2016.
- [16] M. Banoula, “What is perceptron: A beginners guide for perceptron,” 2023. [Online]. Available: <https://www.simplilearn.com/tutorials/deep-learning-tutorial/perceptron>
- [17] A. Geron, *Neural Networks and Deep Learning, in Hands-on machine learning with scikit-learn, Keras, and tensorflow: Concepts, tools and techniques to build Intelligent Systems*. O’Reilly, 2019.
- [18] “Perceptrons and multi-layer perceptrons,” 2022. [Online]. Available: <https://indiantechwarrior.com/perceptrons-and-multi-layer-perceptrons/>
- [19] R. Sharma, “Deep learning activation functions their mathematical implementation,” 2022. [Online]. Available: <https://medium.com/nerd-for-tech/deep-learning-activation-functions-their-mathematical-implementation-b620d536d39b>
- [20] N. S. et al, “Dropout: a simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.

- [21] J. Brownlee, “How to choose an activation function for deep learning,” *MachineLearningMastery.com*, 1 2021. [Online]. Available: <https://machinelearningmastery.com/choose-an-activation-function-for-deep-learning/>
- [22] S. Saha, *Medium*, 2023. [Online]. Available: <https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>
- [23] L. S. Priyono, “Student notes: convolutional neural networks (cnn) introduction,” Apr 2018. [Online]. Available: <https://indoml.com/2018/03/07/student-notes-convolutional-neural-networks-cnn-introduction/>
- [24] A. S. et al, “Digital twin for intelligent context-aware iot healthcare systems,” *A survey on deep-learning-based diabetic retinopathy classification*, vol. 13, no. 3, p. 345, 2023.
- [25] N. T. et al, “Deep learning for diabetic retinopathy detection and classification based on fundus images: A review,” *Computers in Biology and Medicine*, vol. 135, p. 104599, 2021.
- [26] H. C. et al, “Detection of diabetic retinopathy using deep neural network,” *IEEE Internet of Things Journal*, 2018.
- [27] M. Vijayan and V. S, “A regression-based approach to diabetic retinopathy diagnosis using efficientnet,” *Diagnostics*, 2023.
- [28] Z. Zhang, “Deep-learning-based early detection of diabetic retinopathy on fundus photography using efficientnet,” in *Proceedings of the 2020 the 4th International Conference on Innovation in Artificial Intelligence*, 2020.
- [29] M. S. H. et al, “An automated model using deep convolutional neural network for retinal image classification to detect diabetic retinopathy,” in *Proceedings of the International Conference on Computing Advancements*, 2020.
- [30] H. T. D. U. Qomariah and C. Fatichah, “Classification of diabetic retinopathy and normal retinal images using cnn and svm,” in *2019 12th International Conference on Information Communication Technology and System (ICTS)*, 2019.
- [31] H. K. et al, “Diabetic retinopathy diagnosis from fundus images using stacked generalization of deep models,” *IEEE Access*, vol. 9, pp. 108 276–108 292, 2021.

- [32] C. L. et al, “Deep convolutional neural networks for diabetic retinopathy classification,” *Proceedings of the 2nd International Conference on Advances in Image Processing*, 2018.
- [33] F. W. H. Chen, X. Zeng and W. Ye, “Detection of diabetic retinopathy using deep neural network,” *IEEE*, 2018.
- [34] A. S. M. S. V. Chilukoti, L. Shan and X. Hei, “A reliable diabetic retinopathy grading via transfer learning with quadratic weighted kappa metric,” *Research Square*, 2 2023.
- [35] S. Rajamani and S. Sasikala, “Artificial intelligence approach for diabetic retinopathy severity detection,” *Informatica*, vol. 46, no. 8, 1 2023.
- [36] S. A. A. M. Mutawa and S. Sruthi, “Transfer learning for diabetic retinopathy detection: A study of dataset combination and model performance,” *Applied Sciences*, vol. 13, no. 9, p. 5685, 2023.
- [37] K. Doyle, “Frameworks, libraries and languages for machine learning,” *App Architecture*, vol. 13, 5 2021. [Online]. Available: <https://www.techtarget.com/searchapparchitecture/tip/Frameworks-libraries-and-languages-for-machine-learning>
- [38] D. R. D. . Kaggle. [Online]. Available: <https://www.kaggle.com/c/diabetic-retinopathy-detection>
- [39] “Aptos 2019 blindness detection, kaggle.” [Online]. Available: <https://www.kaggle.com/c/aptos2019-blindness-detection>
- [40] M. H. et al, “Impact of clahe-based image enhancement for diabetic retinopathy classification through deep learning,” *Procedia Computer Science*, vol. 216, p. 57–66, 1 2023.
- [41] Y. Makhoulf and A. Daamouche, “Automatic generation of adaptive structuring elements for road identification in vhr images,” *Expert Systems with Applications*, vol. 119, p. 342–349, 2019.
- [42] M. Tan, “Efficientnet: Rethinking model scaling for convolutional neural networks,” 5 2019. [Online]. Available: <https://arxiv.org/abs/1905.11946>
- [43] C. Imane, “Efficientnet [model architecture],” OpenGenus IQ: Computing Expertise Legacy. Available at <https://iq.opengenus.org/efficientnet/>.

- [44] C. Szegedy, “Rethinking the inception architecture for computer vision,” 2 2015. [Online]. Available: <https://arxiv.org/abs/1512.00567>
- [45] A. Brital, “Inception v3 cnn architecture explained,” 1 2022. [Online]. Available: <https://medium.com/@AnasBrital98/inception-v3-cnn-architecture-explained-691cfb7bba08>
- [46] F. Chollet, “Xception: Deep learning with depthwise separable convolutions,” *arXiv.org*, 10 2016. [Online]. Available: <https://arxiv.org/abs/1610.02357>
- [47] S. Tsang, “Review: Xception — with depthwise separable convolution, better than inception-v3 (image classification),” *Medium*, p. 5685, 3 2019.
- [48] M. Z. A. Z. M. Sandler, A. W. Howard and L.-C. Chen, “Mobilenetv2: Inverted residuals and linear bottlenecks,” 2018.
- [49] T. Z. X. W. J.-Y. Y. W. Wang, Y. Li and Y. Luo, “A novel image classification approach via dense-mobilenet models,” *Mobile Information Systems*, pp. 1–8, 2020.

Appendix

Bright and Dark images filtering function

```
import cv2
import numpy as np

def is_bright(image_path, dim=227, brightness_thresh=0.55, darkness_thresh=0.165):
    # Read the image
    image = cv2.imread(image_path)
    # Resize image to dim x dim
    image = cv2.resize(image, (dim, dim))
    # Convert color space to LAB format and extract L channel
    L, _, _ = cv2.split(cv2.cvtColor(image, cv2.COLOR_BGR2LAB))
    # Normalize L channel by dividing all pixel values with the maximum pixel value
    L = L / np.max(L)
    # Check if the mean brightness exceeds the brightness threshold
    if np.mean(L) > brightness_thresh:
        return True
    # Check if the mean brightness is below the darkness threshold
    elif np.mean(L) < darkness_thresh:
        return True
    else:
        return False

def filter_brightness(df):
    filtered_df = df.copy()
    indices_to_drop = []
    for index, row in df.iterrows():
        if is_bright(row['path']):
            indices_to_drop.append(index)
    filtered_df = filtered_df.drop(indices_to_drop)
    filtered_df = filtered_df.reset_index(drop=True)
    return filtered_df
```

Image preprocessing function

```
def image_preprocessing(img):  
  
    img = img.astype(np.uint8)  
  
    # Extract the RGB channels of the image  
    b, g, r = cv2.split(img)  
  
    # Apply CLAHE extracted image  
    clh = cv2.createCLAHE(clipLimit=4.0, tileGridSize=(8,8))  
    r = clh.apply(r)  
    g = clh.apply(g)  
    b = clh.apply(b)  
  
    # Fuse the channels  
    merged_bgr_green_fused = cv2.merge((b, g, r))  
  
    # Apply Gaussian blur to remove noise  
    img_blur = cv2.GaussianBlur(merged_bgr_green_fused, (5,5), 0)  
  
    # Remove the isolated pixels using morphological cleaning operation.  
    kernel1 = np.ones((1, 1), np.uint8)  
    morph_open = cv2.morphologyEx(img_blur, cv2.MORPH_OPEN, kernel1)  
  
    # Applying morph_open operation  
    kernel2 = np.ones((2, 2), np.uint8)  
    morph_open2 = cv2.morphologyEx(morph_open, cv2.MORPH_OPEN, kernel2)  
  
    return morph_open2.astype("float64")
```

Circular cropping

```
import cv2
import numpy as np

def preprocess_image(img_path):
    def crop_image_from_gray(img, tol=7):
        if img.ndim == 2:
            mask = img > tol
            return img[np.ix_(mask.any(1), mask.any(0))]
        elif img.ndim == 3:
            gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
            mask = gray_img > tol
            channels = [img[:, :, i][np.ix_(mask.any(1), mask.any(0))] for i in range(3)]
            img = np.stack(channels, axis=-1)
        return img

    def circle_crop(img):
        img = crop_image_from_gray(img)
        height, width, _ = img.shape
        largest_dimension = np.max((height, width))
        circle_img = np.zeros((height, width), np.uint8)
        cv2.circle(circle_img, (width // 2, height // 2), largest_dimension // 2, 1, thickness=-1)
        img = cv2.bitwise_and(img, img, mask=circle_img)
        return crop_image_from_gray(img)

    img = cv2.imread(img_path)
    img_preprocessed = circle_crop(img)
    return img_preprocessed
```